IEEE Open Journal of
Intelligent Transportation
Systems

Received 27 October 2024; revised 26 January 2025 and 26 April 2025; accepted 23 May 2025. Date of publication 29 May 2025;
date of current version 16 June 2025.

Digital Object Identifier 10.1109/OJITS.2025.3574746

Aircraft Trajectory Segmentation-Based Contrastive Coding: A Framework for Self-Supervised Trajectory Representation

THAWEERATH PHISANNUPAWONG, JOSHUA J. DAMANIK (Graduate Student Member, IEEE), AND HAN-LIM CHOI (Senior Member, IEEE)

Department of Aerospace Engineering, Korea Advanced Institute of Science and Technology, Daejeon 34141, Republic of Korea

CORRESPONDING AUTHOR: H.-L. CHOI (e-mail: hanlimc@kaist.ac.kr)

This work was supported by the Korea Agency for Infrastructure Technology Advancement (KAIA) Grant funded by the Ministry of Land, Infrastructure and Transport under Grant 22DATM-C163373-02. The work of Thaweerath Phisannupawong was supported by the Hyundai Motor Chung Mong-Koo Foundation Global Scholarship.

ABSTRACT Air traffic trajectory recognition has gained significant interest within the air traffic management community, particularly for fundamental tasks such as classification and clustering. This paper introduces Aircraft Trajectory Segmentation-based Contrastive Coding (ATSCC), a self-supervised time series representation learning framework designed to capture semantic information in air traffic trajectory data, resulting in a data representation that best explains the trajectory instances. The framework leverages the segmentable characteristic of trajectories and ensures consistency within the self-assigned segments. Intensive experiments were conducted on datasets from three different airports, totaling four datasets, to compare the performance of the learned representation in downstream classification and clustering with that of other state-of-the-art representation learning techniques. The results show that ATSCC outperforms the baseline methods by aligning the representation with maneuvering procedures. Moreover, ATSCC is adaptable to various airport configurations and applicable to incomplete trajectories. This research has expanded upon existing capabilities, achieving these improvements independently without predefined inputs such as airport configurations, maneuvering procedures, or labeled data. The trajectory datasets used in this paper are available at huggingface.co/datasets/petchthwr/ATFMTraj. The implementation code is publicly available at github.com/petchthwr/ATSCC.

INDEX TERMS Aircraft trajectory, contrastive learning, representation learning, self-supervised learning, time series.

I. INTRODUCTION

AIR TRANSPORTATION plays a crucial role in the world economy; as economic growth increases, so does air traffic. Operational concepts have been developed to accommodate this continuous growth, for the current air traffic management (ATM) system and procedures will not be able to maintain the level of safety and efficiency in handling the increasing traffic, especially for airports with dense traffic flow and complex maneuvering patterns [1], [2]. Thus, airspace modernization has become a challenging aim for the ATM society and the regulatory authorities. The ATM concept is expected to shift from a clearance-based to a trajectory-based operation (TBO), which allows more flexible clearance and efficiency in utilizing airspace for high-density traffic. Although the trajectory is human-interpretable, managing higher density and more diverse patterns can be overwhelming for human operators. Therefore, intelligent management and recognition systems have been developed to assist air traffic controllers in both decision-making and managing flight traffic.

Nowadays, a vast amount of aircraft trajectory data, including Automatic Dependent Surveillance-Broadcast (ADS-B) recording, is publicly available. This availability holds significant potential for trajectory recognition research. Classification and clustering are frequently explored in the literature as fundamental recognition tasks. For instance,

The review of this article was arranged by Associate Editor Wei Ma.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

classification helps air traffic controllers organize trajectories for purposes such as estimating the hourly capacity of airspace [3], [4] and detecting conflicts [5]. On the other hand, clustering has gained attention primarily due to the lack of labels in public trajectory datasets and has been used for traffic characterization and categorization, as can be traced back to [6]. However, the trajectories as time series are typically rich in information, complex, and highly dimensional. Many algorithms designed for these downstream tasks are vulnerable to the high dimensionality of the data and usually necessitate feature extraction. Transforming them into a more generalizable data representation is known to be an effective method for enhancing downstream recognition task performance.

Self-supervised contrastive representation learning has demonstrated effectiveness in enhancing recognition tasks across various applications, including vision, natural language processing, and time series. Although this approach has been extensively applied to real-world time series data, including medical signals, electromagnetics, sound waves, and more, the exploration of moving object trajectories still offers considerable opportunities for exploration. Existing time series contrastive representation learning frameworks often struggle to capture meaningful representations for air traffic trajectory data, as they overlook the understanding of the data from the ATM perspective, leading to misaligned contrastive learning. To address this challenge, this paper introduces Aircraft Trajectory Segmentation-based Contrastive Coding (ATSCC), a self-supervised contrastive representation learning framework designed for aircraft trajectory data motivated by the operational contextualization of ATM. An assumption has been made that the states within a trajectory segment share identical contexts. Thus, ATSCC put together the representation within the same segment while distancing them from others, resulting in a representation that reflects the comprehension of patterns. The results demonstrate that our method outperforms existing approaches in self-supervised time series representation learning for trajectory classification and clustering tasks, resolving various utilization issues. This paper's contributions can be listed as follows:

- To our knowledge, we proposed, for the first time, a self-supervised contrastive time series representation learning framework for aerospace trajectory.
- We propose an effective similarity concept within trajectory data using segmentation from the iterative Ramer-Douglas-Peucker (RDP) algorithm. We have integrated the capabilities of a causal language model to generate a representation where each timestep collects all preceding information, making it suitable for real-time monitoring and tracing incomplete and variable-length trajectories.
- We introduced labeled trajectory datasets essential for assessing classification accuracy and evaluating fidelity of clustering results to the labels, an often-overlooked aspect in prior works.
- ATSCC has proven exceptionally suitable for aircraft trajectory data, outperforming state-of-the-art baselines. The model has overcome limitations associated with locational interpretation by enabling analysis with the semantic representation.

II. RELATED WORKS

A. TIME SERIES REPRESENTATION LEARNING

Several studies have proposed models that generate time series representation vectors by leveraging the similarity among samples. Random Warping Series (RWS) [7] demonstrated a kernel method based on Dynamic Time Warping (DTW) [8], which generates representations through random feature approximation. SPIRAL [9] proposed a method that converts a set of time series into vectors by solving non-convex and non-deterministic polynomial-time hard optimization, preserving the pairwise similarity of instances.

Autoencoders are the most widely applied representation learning for the aerospace domain. An encoder compresses a data instance into a representation vector, which is subsequently used by the decoder to reconstruct the original input instance. The model is trained with a reconstruction loss formulated using arbitrary differentiable distance functions and can be constructed with various architectures such as multi-layer perceptrons (MLP) [10], [11], convolutional neural networks (CNN) [12], [13], recurrent neural networks (RNN) [14], long-short term memory (LSTM) [15], [16], dilated causal convolutional networks [17], as implemented by [18], [19], or transformers [20] as in [21].

Contrastive learning optimizes the encoded representation in the embedding space, ensuring the positive pairs or groups are similar in the embedding space while being distinct from the negative ones. The main idea of developing a contrastive representation learning framework is to establish an effective definition of positive and negative samples, along with an appropriate neural network architecture and training strategy. T-loss [22] employs a triplet loss for training, where a positive sample is obtained from a sub-series within the random reference area of an instance, and the negatives are all other sub-series. TNC [23] leverages the local smoothness of the time series to model a positive temporal neighborhood, while the negatives are sub-series that are temporally distant. TNC maximizes similarity likelihood within the neighborhood while minimizing that of the negatives. TS-TCC [24] maximizes the agreement between two augmented views of an instance while setting them apart from other samples. TS2Vec [25] employs random cropping and binomial masking for data augmentation. The encoder is trained by applying the hierarchical contrastive loss on the embeddings of two cropped time series, both temporally and instance-wise, at every temporal max pooling level. InfoTS [26] incorporates meta-learning techniques to identify the most suitable time series augmentation method. The encoder is trained using

InfoNCE loss, contrasting temporally and instance-wise, while the meta-learner learns by balancing fidelity and variety loss, similar to [27]. Contrastive learning enables us to define semantic similarities and rules for elements in data, thereby aligning data representation more closely with human contextual understanding.

B. TRAJECTORY CLASSIFICATION

In the transportation domain, trajectory classification has been applied to various tasks, such as crash prediction using a transformer-conformer model [28] trained on recorded crash data, and speeding level classification using rule-based labels with a Support Vector Machine (SVM) classifier [29]. In the ATM domain, one of the fundamental challenges in trajectory recognition is runway classification [30]. The study demonstrated the classification of landing trajectories using a labeled dataset, comparing various classifiers. However, recent works on air traffic patterns are fewer than those on clustering, as public trajectory data are often unlabeled. Efforts in data labeling have been demonstrated in several works, where clustering results are analyzed and utilized as labels to train classifiers, such as the Random Forest algorithm [3], [4], [5], MLP [31], and LSTM [32]. Enforcing a classification loss such as cross-entropy tends to result in representations that are close to one-hot vectors. Moreover, these works emphasize the necessity of labeled datasets.

C. TRAJECTORY CLUSTERING

Trajectory clustering is widely adopted for pattern recognition in the transportation domain, where many methods represent trajectory features as time series [33]. Most simply, trajectories can be resampled to a fixed length and then clustered using Euclidean distance, as demonstrated in [3], [4], [5], [34], [35], or using weighted Euclidean distance, as shown in [36]. For variable-length trajectory datasets, clustering can be performed using a matrix of time series pairwise distances. Examples include DTW combined with agglomerative clustering [37] as in [6], [32], route similarity [38] with progressive clustering [39] using OPTICS [40] as demonstrated in [41], Symmetrized Segment-Path Distance (SSPD) [42] with HDBSCAN [43] on simplified trajectories using the RDP algorithm [44], [45] as shown in [34], and DTW combined with K-medoids [46], dependent on comparisons to human-specified standard procedures, as in [47]. Aircraft flight paths can also be a feature for clustering. For example, DTW combined with HDBSCAN [43] was used on track angle sequences [48]. In [49], track sequences were modeled using Von Mises distributions and clustered with K-medoids using Bhattacharyya distance [50].

The partitioning-based method can be traced back to TRACLUS [51], which identifies points where states change rapidly for partitioning purposes. It utilizes DBSCAN [52] with line segment distance to cluster line segments and then combines these segments to form clusters. Similarly, the methods in [53], [54] identify turning points based on heading changes and group those points using K-means [55] and DBSCAN. These approaches then form the clusters by aggregating the sequences of waypoints from a dependency tree. These methods often rely on spatial interpretations and frequently yield a large number of clusters. The hidden Markov model in [56] converts trajectories into string sequences of turning actions, which are then clustered using K-medoids with the edit distance.

Clustering on raw samples can be inefficient with high-dimensional samples. Many works have demonstrated clustering on dimensionality-reduced samples, such as PCA [13], [53], [57], and t-SNE [12], [58]; however, these methods are typically used for visualization. Moreover, they focus on preserving the characteristics of the dataset rather than those of individual data instances. For representation-based methods, autoencoders are commonly used, and the trained encoder's output serves as the trajectory representation; for example, the MLP with Gaussian Mixture Model (GMM) [59] in [10] or with K-Means in [11], and the CNN for visual representation with GMM as in [13]. Deep clustering is typically an extension of the representation-based methods, often utilizing either a pretrained feature extractor or learning the representation concurrently with the training process. A popular framework for trajectory deep clustering is Deep Embedded Clustering (DEC) [60], implemented with various architecture choices such as CNN in [12], bidirectional LSTM in [15], and time and attribute LSTM (TA-LSTM), an LSTM with a time regulator gate for irregular time intervals, as in [16]. For both clustering on representations and deep clustering, having a high-fidelity representation can enhance performance by effectively converging into distinct classes.

III. METHODOLOGY

A. PROBLEM DEFINITION

For N instances of time series in a variable length trajectory dataset, curated from surveillance data, a multivariate time series, denoted as $\mathcal{X}_i = \{x_{i,1}, x_{i,2}, x_{i,3}, \ldots, x_{i,T_i}\}$, has a dimension of $T_i \times F$, where T_i is the sequence length of ith sample and F is the number of features which varies based on the selected geometric features. In detail, $x_{i,t}$ initially consists of position $\{x^x_{i,t}, x^y_{i,t}, x^z_{i,t}\}$ in the East-North-Up (ENU) coordinates. In this paper, $x_{i,t}$ can be extended to 9 features, incorporating directional vectors $\{x^{u_x}_{i,t}, x^{u_y}_{i,t}, x^{u_z}_{i,t}\}$ and polar components $\{x^r_{i,t}, x^{\sin\theta}_{i,t}, x^{\cos\theta}_{i,t}\}$. It should be noted that the sequence length T_i mentioned is the explicit time, and T_i varies among instances as we deliberately avoided interpolating the trajectory data to a uniform length.

The encoder $f_w(\mathcal{X}_i)$, with learnable parameters w, is trained to best describe $\mathcal{X}_i$ with the representation $\mathcal{Z}_i = \{z_{i,1}, z_{i,2}, z_{i,3}, \ldots, z_{i,T_i}\}$, that has a dimension of $T_i \times K$. Each $z_{i,t} \in \mathbb{R}^K$ summarizes the subseries $\mathcal{X}'_{i,t} = \{x_{i,k} : k \leq t\}$, where $\mathcal{X}'_{i,t} \subseteq \mathcal{X}_i$ comprises the first state $x_{i,1}$ up to $x_{i,t}$. In this paper, a segment is defined as an interval on the time axis that covers states between two consecutive points in the set of approximated turning points, including the beginning and ending points. The training of the encoder $f_w(\mathcal{X}_i)$ involves

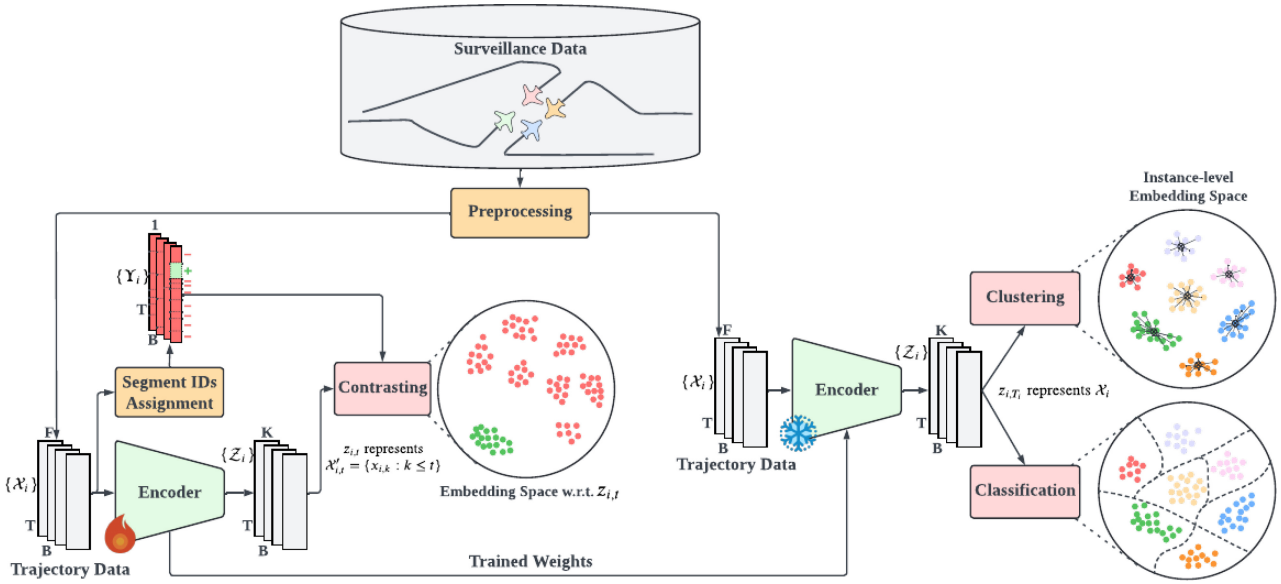


FIGURE 1. Problem formulation of ATSCC, consisting of (left) a self-supervised training phase involving the assignment of segment ID Υ_i to mark $v_{i,t}$ on the subseries $\mathcal{X}'_{i,t} = \{x_{i,k} : k \le t\}$, using the full trajectory $\mathcal{X}_i$ to contrast the representation $z_{i,t}$ of $\mathcal{X}'_{i,t}$; and (right) an evaluation phase on downstream instance-level tasks, including classification and clustering, where z_{i,T_i} corresponds to the full instance $\mathcal{X}_i \equiv \mathcal{X}'_{i,T_i}$.

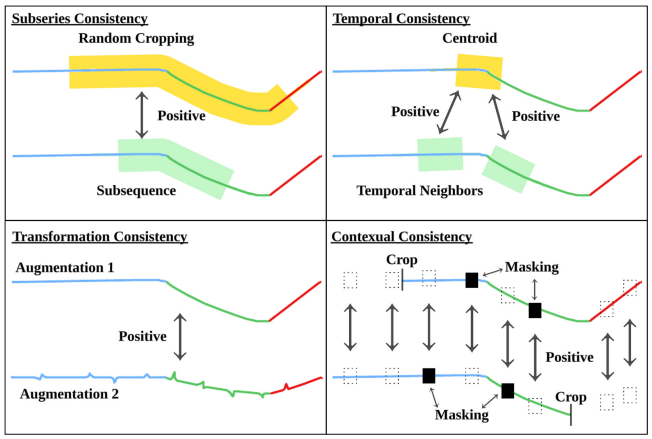


FIGURE 2. Diagram of positive pair selection on a trajectory with three segments in baseline methods: Subseries, temporal, transformation, and contextual consistencies.

the self-assignment of the local segment IDs, using $\mathcal{X}_i$, in a sequence denoted as $\Upsilon_i = \{v_{i,1}, v_{i,2}, v_{i,3}, \ldots, v_{i,T_i}\}$ for $v_{i,t} \in \mathbb{N}$. For each Υ_i on $\mathcal{X}_i$, the set $\{x_{i,t} | v_{i,t} = v\}$ represents $x_{i,t}$ within the same segment v. For $\mathcal{Z}_i$, the $z_{i,t}$ within the same segment are collected in $\{z_{i,t} | v_{i,t} = v\}$ corresponds to $\{\mathcal{X}'_{i,t} | v_{i,t} = v\}$. The representation is learned by contrasting $z_{i,t}$, representing a subseries $\mathcal{X}'_{i,t}$, with Υ_i denoting positive and negative sets. This approach benefits downstream instance-level tasks, including classification and clustering, where z_{i,T_i} represents $\mathcal{X}_i \equiv \mathcal{X}'_{i,T_i}$. The problem formulation of this paper is visualized in Figure 1.

B. CONSISTENCY WITHIN SEGMENTS

Self-supervised contrastive representation learning for time series data typically relies on human-designed similarity concepts, which serve as rules for assigning data samples, whether at the instance, sub-instance, or timestamp levels, to positive and negative sets. In this context, consistency refers to the outcomes of encouraging a similarity concept, which is learned by contrasting the assigned positive and negative sets. This process enables models to effectively group similar samples and distinguish them from those that are semantically different. Previous works have introduced concepts of subseries, temporal, transformation, and contextual consistency as shown in Figure 2. As referenced in [25], [26], [61], [62], the challenges of unsuitable augmentations and similarity concepts are discussed. These issues can cause various methods to potentially underperform for air traffic trajectories due to the misalignments they introduce.

- *Missing Information:* To enforce subsequence consistency, T-Loss [22] defines positive pairs using subsequences, while TS2Vec [25] employs random cropping for contextual consistency. These approaches risk removing critical segments, leading the model to treat incomplete trajectories as equivalent to more advanced ones, such as equating a trajectory not yet in the landing phase with one performing a landing.
- *Incorrect Flight Phase Grouping:* TNC [23] samples positive subseries from temporal neighborhoods to enforce temporal consistency, but may introduce false positives when sampling centroids are near turning points, leading to the grouping of sub-trajectories from different flight phases, such as landing and approach.
- *Procedure Distortion:* Enforcing transformation consistency, as in TS-TCC [24], through permutation, scaling, or jittering can create augmented trajectories that deviate from standard procedures. This may produce positive pairs that no longer reflect the maneuvering

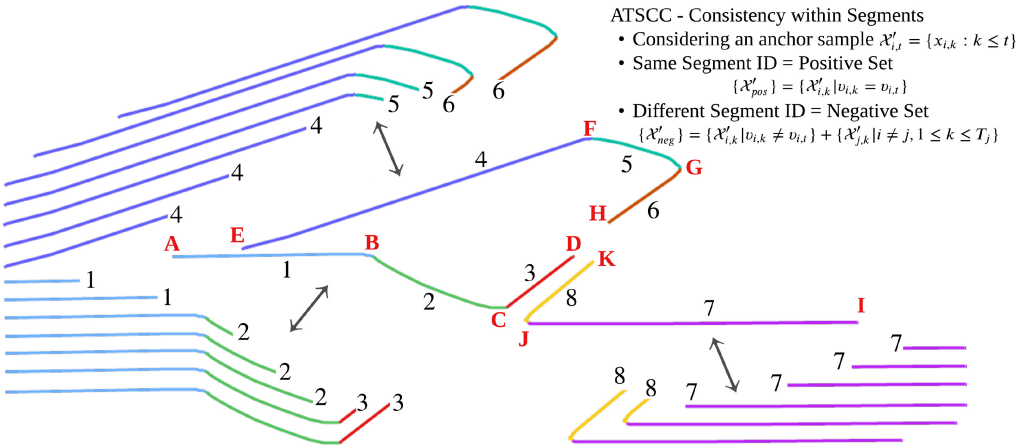


FIGURE 3. Visualization of ATSCC's consistency within segments: Considering a batch of three trajectories $\mathcal{X}i$, each assigned a sequence of segment IDs Υ_i. Consistency within a segment is enforced by selecting $\mathcal{X}'_{i,t}$ with the same segment ID as positive and others as negatives.

procedure, forcing the original trajectory to align with its intensively transformed version.

- *Combined Misalignment:* While InfoTS [26] combines all the methods mentioned with additional augmentations like time warping, window warping, cutout, and slicing, and can identify the best augmentation for specific types of time series, it still faces misalignments. InfoTS inherits issues from the four contrastive methods and introduces procedure distortion through these additional transformation choices.

Therefore, we need to establish a similarity concept with specific rules tailored to aircraft trajectories in terminal airspace. Our approach is motivated by the segmenting behavior observed in many air traffic control (ATC) tasks. For example, aircraft vectoring instructions remain unchanged until the aircraft reaches a designated turning position or is given a new instruction. In waypoint navigation, a sequence of waypoints instructs the aircraft operator to comply, forming the trajectory. In TBO, planned trajectories are divided by detailed checkpoints. Inspired by these tasks, we infer the semantic context within segments partitioned by significant positions. The contribution of our framework lies in our unique method of assigning positive and negative samples, using the geometric properties of trajectories rather than data augmentation, avoiding the aforementioned misalignments. We propose ATSCC (Figure 4), a framework that enforces consistency within segments, as visualized in Figure 3, using an example batch of three trajectories segmented by the displayed waypoints and well-assigned segment IDs.

In the ATSCC framework, we consider an anchor sub-trajectory $\mathcal{X}'_{i,t}$ marked with a segment ID $\upsilon_{i,t}$, one at a time. Sub-trajectories that have captured all previous states and reached the same waypoint but have not yet reached the next waypoint, denoted as $\{\mathcal{X}'_{i,k} \mid \upsilon_{i,k} = \upsilon_{i,t}\}$, are designed to share the same representation. Sub-trajectories that have not reached the waypoint or have passed the next waypoint, denoted $\{\mathcal{X}'_{i,k} \mid \upsilon_{i,k} \neq \upsilon_{i,t}\}$, should have a different representation to the sub-trajectory $\mathcal{X}'_{i,t}$. This ensures that sub-trajectories following the same maneuvering phase are represented consistently, thereby preventing incorrect grouping of flight phases. For intuition, in a trajectory navigating through waypoints ($A \to B \to C \to D$), the sub-trajectory that has passed waypoint B but not yet reached waypoint C should have the same representation, while differing from the sub-trajectory that has not yet passed waypoint B or has already passed waypoint C. Additionally, these sub-trajectories should be represented differently from those in any other trajectory instances in the batch, denoted $\{\mathcal{X}'_{j,k} | i \neq j, 1 \le k \le T_j\}$ to clearly distinguish different maneuvering procedures. For illustrations, we distinguish all sub-trajectories of the trajectory navigating ($A \to B \to C \to D$) from those in the trajectory navigating ($E \to F \to G \to H$) and those in the trajectory navigating ($I \to J \to K$). We illustrate this similarity concept in Figure 3, where the positive and negative sets of a sub-trajectory $\mathcal{X}'_{i,t}$ are represented by different IDs and are precisely defined as:

$$\left\{\mathcal{X}'_{pos}\right\} = \left\{\mathcal{X}'_{i,k} | \upsilon_{i,k} = \upsilon_{i,t}\right\} \tag{1}$$

$$\left\{\mathcal{X}'_{neg}\right\} = \left\{\mathcal{X}'_{i,k} | \upsilon_{i,k} \neq \upsilon_{i,t}\right\} + \left\{\mathcal{X}'_{j,k} | i \neq j, 1 \le k \le T_j\right\} \tag{2}$$

C. SEGMENT ID ASSIGNMENTS

The ATSCC framework aims to group together the representations at timestamps within a segment while distancing them from those at timestamps outside that segment and from those within other instances. Thus, a systematic assignment of segment IDs is needed as a mapping to enforce this similarity concept. When waypoints are not well-defined, as in Figure 3, for a self-supervised setting, trajectories can be segmented into significant points [53], [54], which are assumed to be close enough to the instructed waypoints, thus forming segments within the trajectory. The RDP algorithm [44], [45] reduces points in a curve while preserving its essential shape and geometric integrity, resulting in a simplified curve with timestamps of significant points

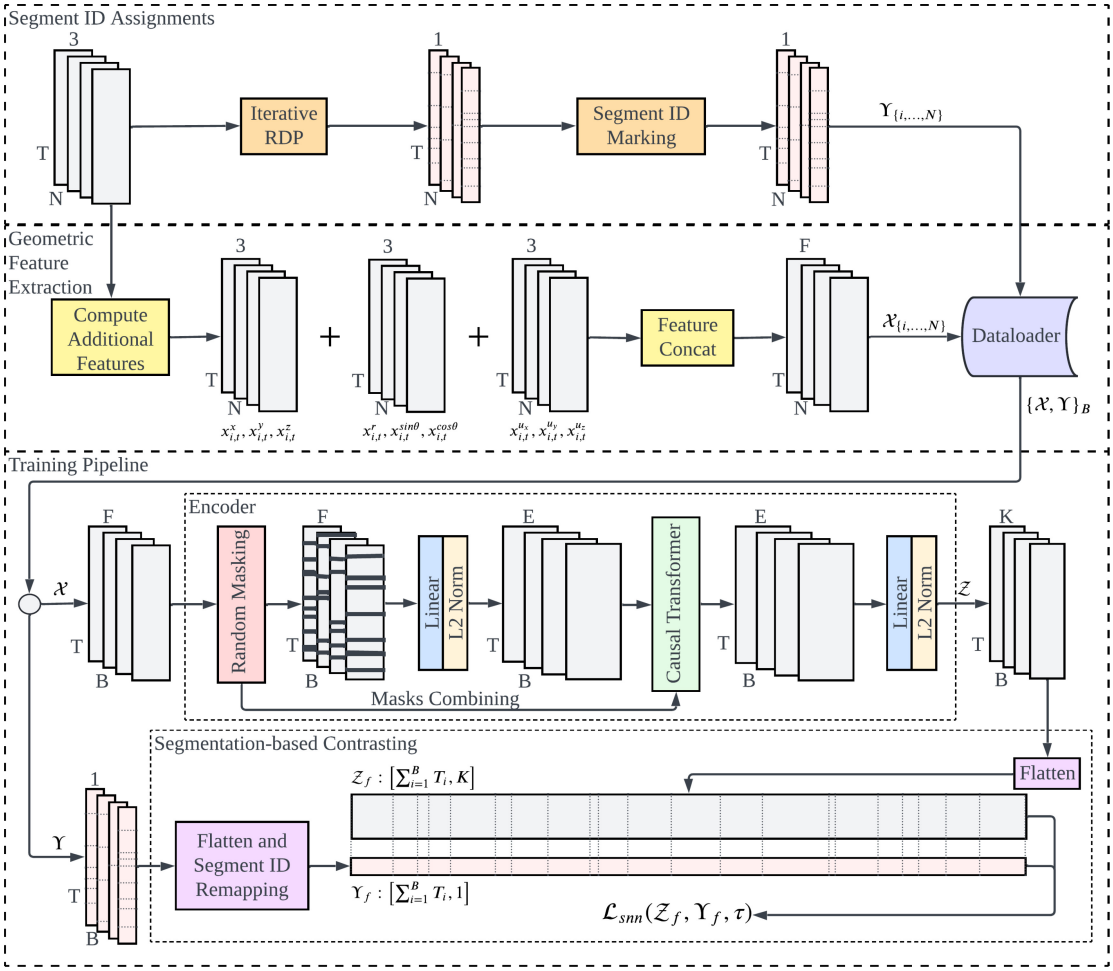


FIGURE 4. Diagram of the proposed framework for aircraft trajectory segmentation-based contrastive coding (ATSCC): Includes segment ID assignments using the iterative RDP algorithm, geometric feature extraction, training pipeline with a causal transformer encoder, ID Remapping, and training with soft-nearest neighbor loss.

that segment the curve. We used an open-source Python library for the iterative RDP algorithm [63], as described in Algorithm 1. It starts by marking the beginning and end points of the curve as significant. Then, for each segment between significant points, the algorithm identifies points that exceed a perpendicular distance threshold, ϵ, from the line segment and marks these points as significant. This process repeats until no further points exceed the threshold. The perpendicular distance from a point $x_{i,k}$ to a line segment defined by points $x_{i,s}$ and $x_{i,e}$, is denoted as $d(x_{i,k}, x_{i,s}, x_{i,e})$ and is given by:

$$d\big(x_{i,k}, x_{i,s}, x_{i,e}\big) = \begin{cases} \|x_{i,k} - x_{i,s}\| & \text{if } x_{i,s} = x_{i,e}, \\ \frac{\|(x_{i,e} - x_{i,s}) \times (x_{i,s} - x_{i,k})\|}{\|x_{i,e} - x_{i,s}\|} & \text{otherwise.} \end{cases} \quad (3)$$

Here, s, k, and e denote the indices of the start, considering, and end points of the trajectory segment, respectively. $x_{i,s}$, $x_{i,k}$ and $x_{i,e}$ refer to thier corresponding points within $\mathcal{X}_i$. Note that for this algorithm, $x_{i,t} = \{x^x_{i,t}, x^y_{i,t}, x^z_{i,t}\}$. The iterative RDP algorithm produces the mask $M_i = \{m_{i,1}, m_{i,2}, m_{i,3}, \ldots, m_{i,T}\}$, where each $m_{i,t} \in \{0, 1\}$. An instance's segment IDs $\Upsilon_i = \{\upsilon_{i,1}, \upsilon_{i,2}, \upsilon_{i,3}, \ldots, \upsilon_{i,T}\}$ is obtained from the cumulative sum by each segment ID $\upsilon_{i,t} = \sum_{k=1}^{t} m_{i,k}$ and $\upsilon_{i,T} = \upsilon_{i,T-1}$ where $\upsilon_{i,t} \in \mathbb{N}$ is marked corresponded to the temporal indices, t on $x_{i,t}$.

The process of segment ID assignment can be described as a function where $\Upsilon_i = RDP(\mathcal{X}_i, \epsilon)$, indicating that Υ_i is the result of performing segment ID assignment via the RDP algorithm with an allowable perpendicular line segment distance of ϵ to the trajectory $\mathcal{X}_i$. ϵ serves as a measure of the allowable error between $\mathcal{X}_i$ and its simplified version. Thus, changes in ϵ alter how the operational context boundary is defined, so ϵ is taken as a hyperparameter of the ATSCC. Each Υ_i is marked locally on each $\mathcal{X}_i$, independent of other samples. All Υ_i for N instances of $\mathcal{X}_i$ are precomputed for computational efficiency. They are padded with NaN to $T_{\max} = \max_{i \in 1 \ldots N}(T_i)$, in the same manner as the unequal length trajectory dataset, and stored within the PyTorch Dataloader module for later use in the training pipeline.

D. GEOMETRIC FEATURE EXTRACTION

The trajectory data describe the sequence of aircraft's positions in ENU coordinates as $\mathcal{X}_i \in \{x_{i,1}, x_{i,2}, x_{i,3}, \ldots, x_{i,T}\}$; each $x_{i,t}$ initially consists of $x^x_{i,t}, x^y_{i,t}, x^z_{i,t}$. To extend the

Algorithm 1 Local Segment ID Marking via Iterative Ramer-Douglas-Peucker Algorithm

procedure RDP($\mathcal{X}_i$, ϵ)
  // Step 1 : Iterative RDP Algorithm [63]
  $\mathcal{S} \leftarrow \{(1, T_i)\}$
  $M_i \leftarrow \{1 : k = 1, \ldots, T_i\}$
  $n_{\mathcal{S}} = |\mathcal{S}|$
  while $n_{\mathcal{S}} \neq 0$ **do**
    $(s, e) \leftarrow \mathcal{S}[n_{\mathcal{S}}]$
    $\mathcal{S} \leftarrow \mathcal{S} - \{(s, e)\}$
    $d_{max} \leftarrow 0.0$
    $t \leftarrow s$
    for $k \leftarrow t + 1$ to $t_e - 1$ **do**
      if $m_{i,k}$ **then**
        $d \leftarrow d(x_{i,k}, x_{i,s}, x_{i,e})$
        if $d > d_{max}$ **then**
          $t \leftarrow k$
          $d_{max} \leftarrow d$
        end if
      end if
    end for
    if $d_{max} > \epsilon$ **then**
      $\mathcal{S} \leftarrow \mathcal{S} + \{(s, t), (t, e)\}$
    else
      for $k \leftarrow s + 1$ to $e - 1$ **do**
        $m_{i,k} \leftarrow 0$
      end for
    end if
    $n_{\mathcal{S}} = |\mathcal{S}|$
  end while
  // Step 2: Local segment ID assignment
  $\Upsilon_i \leftarrow \{\}$
  for t in T **do**
    if $t \neq T$ **then**
      $\upsilon_{i,t} = \sum_{k=1}^{t} m_{i,k}$
    else
      $\upsilon_{i,t} = \sum_{k=1}^{t-1} m_{i,k}$
    end if
    $\Upsilon_i \leftarrow \Upsilon_i + \{\upsilon_{i,t}\}$
  end for
  return Υ_i
end procedure

underlying information in trajectory states, this feature extraction calculates additional geometric features prior to the training stage. Since contrastive representation learning can handle features of different scales, these features enhance information without typical scaling issues found in feature-based methods or autoencoders. Empirically, this inclusion improves performance, as discussed in the ablation study.

Aircraft path angle is a useful feature for representing trajectory shape [11], [48], [49]. Moreover, most ATM instructions involve path angles, such as headings, glide paths, or directing aircraft between waypoints and specific altitudes. In other words, ATC uses not only the aircraft's position but also its direction. This paper utilizes the directional unit vector to represent aircraft paths, thereby avoiding trigonometric singularities associated with angles. The aircraft's path vector at each timestamp is given by

$$\left\{x_{i,t}^{u_x}, x_{i,t}^{u_y}, x_{i,t}^{u_z}\right\} = \frac{x_{i,t+1} - x_{i,t}}{\left\|x_{i,t+1} - x_{i,t}\right\|}, \tag{4}$$

where $\|x_{i,t+1} - x_{i,t}\|$ denotes the Euclidean norm of the vector difference, and $x_{i,t} = \{x_{i,t}^x, x_{i,t}^y, x_{i,t}^z\}$. For the final time step T_i, it is assumed that without further updates, the aircraft maintains the last computed path, such that

$$\left\{x_{i,T_i}^{u_x}, x_{i,T_i}^{u_y}, x_{i,T_i}^{u_z}\right\} = \left\{x_{i,T_i-1}^{u_x}, x_{i,T_i-1}^{u_y}, x_{i,T_i-1}^{u_z}\right\}. \tag{5}$$

Interpreting positions in polar coordinates centered at reference waypoints is crucial for navigational aids such as VOR/DME (Very High-Frequency Omnidirectional Range with Distance Measuring Equipment) stations. The sensitivity of these states increases as the aircraft maneuvers near the reference point, in this case, the airport. This behavior establishes rapid changes in features, providing ease of recognition. The polar components feature at each timestamp can be calculated with,

$$\left\{x_{i,t}^r, x_{i,t}^{\sin\theta}, x_{i,t}^{\cos\theta}\right\} = \left\{\begin{array}{c} \sqrt{\left(x_{i,t}^x\right)^2 + \left(x_{i,t}^y\right)^2} \\ \sin\left(\arctan 2\left(x_{i,t}^y, x_{i,t}^x\right)\right) \\ \cos\left(\arctan 2\left(x_{i,t}^y, x_{i,t}^x\right)\right) \end{array}\right\}^T. \tag{6}$$

We use the sine and cosine of the angle from the arctan 2 function to avoid singularities and discontinuities at 0 and 360 degrees. This method ensures a smooth transition across quadrant boundaries. The features are combined to form $x_{i,t} = \{x_{i,t}^x, x_{i,t}^y, x_{i,t}^z, x_{i,t}^{u_x}, x_{i,t}^{u_y}, x_{i,t}^{u_z}, x_{i,t}^r, x_{i,t}^{\sin\theta}, x_{i,t}^{\cos\theta}\}$. These are then stored in the PyTorch DataLoader module along with the segment IDs Υ_i.

E. TRAINING PIPELINE

1) ENCODER ARCHITECTURE

The encoder f_w consists of a random masking module, a Causal Transformer backbone, and input/output projection layers, as shown in Figure 4. The encoder first applies binomial timestamp masking to the inputs during training, similar to [25]. However, our encoder employs masking before the input projection to mimic missing states caused by irregular transmissions, providing regularization similar to dropout. This random mask is later merged with the source padding mask and causal mask for the attention to ignore the masked tokens.

The input linear projection expands each state $x_{i,t}$ from dimension K to the token dimension E of the encoder backbone. The architecture of the Causal Transformer backbone, illustrated in Figure 5, is similar to the designs of GPT-2 [64] and GPT-3 [65]. For each batch of inputs $\mathcal{X} = \{\mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3, \ldots, \mathcal{X}_B\}$, the encoder handles unequal-length

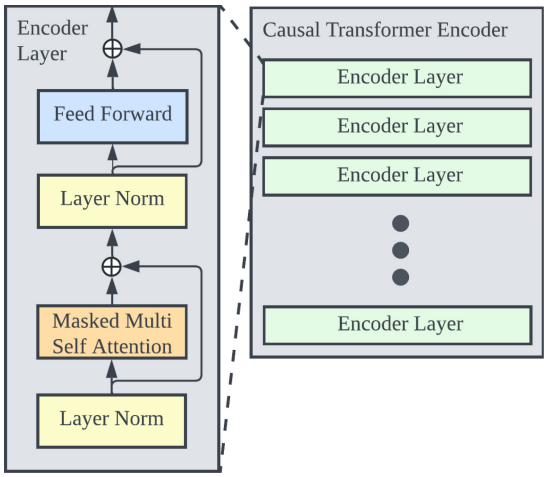


FIGURE 5. Architecture of causal transformer backbone of the encoder.

sequences by applying key padding masks on inputs, outputs, and attention matrices. The causal attention mechanism restricts attention only to previous tokens, ensuring that the output for any position reflects the set encoding of all preceding tokens. Thus, the model's output at any given time, $z_{i,t}$, is the summary of $\mathcal{X}'_{i,t} = \{x_{i,k} : k \leq t\}$. Each encoder state, having a dimension of E, is then linearly projected into $z_{i,t}$ with a representation size of K.

We follow the NoPos approach [66], which shows that the transformer causal language model (CLM) can perceive positional information without explicit positional encoding. Thus, when applied to time series data, the CLM effectively captures their sequential behavior. We apply L2 normalization after each linear projection layer, as suggested in [67] that it can improve performance by making the embeddings linearly separable. Additionally, L2 normalization on tokens before encoding slightly enhances the effectiveness of the learned representation for downstream tasks.

2) SEGMENTATION-BASED CONTRASTING

As the ATSCC framework uses the segment IDs assigned by the RDP algorithm to define the similarity of samples, this contrastive representation learning becomes simple yet effective. First, a batch time series data, $\mathcal{X} = \{\mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3, \ldots, \mathcal{X}_B\}$, is encoded through the encoder f_w, resulting in a batch of representation $\mathcal{Z} = \{\mathcal{Z}_1, \mathcal{Z}_2, \mathcal{Z}_3, \ldots, \mathcal{Z}_B\}$. As $z_{i,t}$ represents $\mathcal{X}'_{i,t} = \{x_{i,k} : k \leq t\}$, the contrasting applied to each element of an encoded batch $\mathcal{Z}$ is equivalent to both temporal and instance-wise contrasting on a set the incomplete instances $\{\mathcal{X}'_{i,t}\}$ for $t \in \{1, 2, 3, \ldots, T_i\}$ and $i \in \{1, 2, 3, \ldots, B\}$. The contrasting on an encoded batch $\mathcal{Z}$ is dictated by the coresponding batch of segment IDs, $\Upsilon = \{\Upsilon_1, \Upsilon_2, \Upsilon_3, \ldots, \Upsilon_B\}$. The framework define positive samples as $z_{i,t}$ within the same segment that shares the same $\upsilon_{i,t}$, while treating other $z_{i,t}$ in different segments and $z_{j,t}$ from other instances $j \in \{1, 2, 3, \ldots, B\}$ with $j \neq i$ as negative samples. The procedure of loss calculation is described in Algorithm 2.

Algorithm 2 Calculation of Soft-Nearest Neighbor Loss

procedure SNNL($\mathcal{Z}$, Υ, τ)
  $\mathcal{Z}_f \leftarrow \{\}$
  $\Upsilon_f \leftarrow \{\}$
  $\upsilon_{\text{current}} \leftarrow 1$
  for $i \leftarrow 1$ to B **do**
    // Flatten $\mathcal{Z}$
    $K \leftarrow \{t | \upsilon_{i,t} \neq \text{NaN}, \upsilon_{i,t} \in \Upsilon_i\}$
    $\mathcal{Z}_f \leftarrow \mathcal{Z}_f + \{z_{i,t} | t \in K, z_{i,t} \in \mathcal{Z}_i\}$
    // Segment ID remapping
    $g_m : \{\} \rightarrow \{\}$
    $\Upsilon_{i,\text{new}} \leftarrow \{\}$
    $\Upsilon_i \leftarrow \{\upsilon_{i,t} | t \in K, \upsilon_{i,t} \in \Upsilon_i\}$
    for $\upsilon_{i,t}$ **in** Υ_i **do**
      if $g_m(\upsilon_{i,t})$ does not exist **then**
        $g_m(\upsilon_{i,t}) \leftarrow \upsilon_{\text{current}}$
        $\upsilon_{\text{current}} \leftarrow \upsilon_{\text{current}} + 1$
      end if
      $\Upsilon_{i,\text{new}} \leftarrow \Upsilon_{i,\text{new}} + \{g_m(\upsilon_{i,t})\}$
    end for
    $\Upsilon_f \leftarrow \Upsilon_f + \Upsilon_{i,\text{new}}$
  end for
  // Calculate soft-nearest neighbor loss
  Loss $= \mathcal{L}_{ssn}(\mathcal{Z}_f, \Upsilon_f, \tau)$; Equation (8)
  return Loss
end procedure

The batch of embedding $\mathcal{Z}$ is flattened into the 2-dimensional $\mathcal{Z}_f$ having the number of features of K and the length of $\sum_{i=1}^{B} T_i$, for T_i is not equal across instances. During the ID assigning (Algorithm 1), each sequence of segment IDs Υ_i is marked locally, starting from 1. The ID remapping on the batch of segment IDs Υ is performed using the updatable mapping function g_m. This procedure ensures that when local segment IDs are flattened, the uniqueness of IDs across segments and instances is guaranteed, as in Figure 3, preventing segments from different samples from being mistakenly identified as positives. The length of the flattened segment IDs, Υ_i is $\sum_{i=1}^{B} T_i$, where each ID $\upsilon_{i,t}$ matches the corresponding $z_{i,t}$. Given the presence of multiple positive samples, the ATSCC framework employs a modified soft-nearest neighbor loss (SNNL) [68], which utilizes scaled vector multiplication instead of distances, as each $z_{i,t}$ has been L2-normalized. The rearranged soft-nearest neighbor loss is given by equation (7).

$$\mathcal{L}_{snn}(\mathcal{Z}_f, \Upsilon_f, \tau) = -\mathop{\mathbb{E}}_{\substack{z\sim\mathcal{Z}_f \\ \upsilon\sim\Upsilon_f \\ i\sim|\mathcal{Z}_f|}} \left(\log \sum_{\substack{j=1 \\ j\neq i \\ \upsilon_i=\upsilon_j}}^{|\mathcal{Z}_f|} e^{\left(\frac{z_i^T z_j}{\tau}\right)} - \log \sum_{\substack{k=1 \\ k\neq i}}^{|\mathcal{Z}_f|} e^{\left(\frac{z_i^T z_k}{\tau}\right)} \right). \quad (7)$$

A single model parameter update step in ATSCC processes a large number of $z_{i,t}$ in one batch; for example, a batch

size of 16 trajectories with an average length of 500 steps results in 8,000 representation vectors in $\mathcal{Z}_f$. We assume that the inclusion of positive contrasting terms in the sum of negative terms has a minimal effect. Moreover, omitting positive contrasts in the negative terms, as in equation (8), leads to empirically better performance.

$$\mathcal{L}_{snn}\left(\mathcal{Z}_f, \Upsilon_f, \tau\right) = -\mathop{\mathbb{E}}_{\substack{z\sim\mathcal{Z}_f \\ \upsilon\sim\Upsilon_f \\ i\sim|\mathcal{Z}_f|}} \left(\log \sum_{\substack{j=1 \\ j\neq i \\ \upsilon_i=\upsilon_j}}^{|\mathcal{Z}_f|} e^{\left(\frac{z_i^T z_j}{\tau}\right)} - \log \sum_{\substack{k=1 \\ k\neq i \\ \upsilon_i\neq\upsilon_k}}^{|\mathcal{Z}_f|} e^{\left(\frac{z_i^T z_k}{\tau}\right)} \right) \quad (8)$$

The loss function maximizes agreement between the representation vectors within the assigned segments while explicitly differentiating them from those in other segments and other instances within the same batch. This modified form ensures that enforcing the soft-nearest neighbor loss supports our fundamental similarity concept described by equation (1) and equation (2); that is, the collective trajectory states within the same operational context area should result in a similar representation clearly distinct from those outside.

IV. RESULTS AND DISCUSSION

In this section, we first elaborate on the dataset preparation used for training and evaluation. Then, we outline the experiment setting for evaluating the learned trajectory representation on fundamental air traffic management downstream tasks, including classification and clustering. We compare the performance of our learned representation with that of state-of-the-art representation learning methods to demonstrate ATSCC's suitability for air traffic data. The feasibility of model components was assessed in the ablation study, and the impact of the core hyperparameters was examined in the sensitivity analysis. Additionally, we provide a detailed visual explanation of the learned representation and then demonstrate ATSCC's robustness to low-quality data. Finally, we present the utilization of ATSCC for trajectory categorization, addressing a common problem in the field.

A. DATASETS

1) DATASET SOURCES

This study utilized trajectory data from three different airports, resulting in a total of four datasets. We aim to demonstrate the broad applicability of the ATSCC framework across various airport configurations. Furthermore, we gathered data from public sources and from other researchers to ensure the framework's applicability, the reliability of the datasets, and the validity of the experiments. The data sources are:

- *Incheon International Airport* (Denoted as ICAO code: RKSI): The ADS-B data were sourced from the OpenSky database [69]. The data for flights from 2018 to 2023 were queried using the flight identification numbers from the Airportal website, [70]. We down-sampled the trajectory data from the southbound and southeastbound flights to improve data balance. The datasets for arrivals and departures are denoted as RKSIa and RKSId, respectively.
- *Stockholm Arlanda Airport* (Denoted as ICAO code: ESSA): The data is from the Swedish Civil Air Traffic Control (SCAT) dataset [71], which includes surveillance data, weather, flight plans, and airspace data. We focused on surveillance data for positional states and flight plans to assist our manual labeling. We only use arrival data for this airport, as the departure data was found incomplete.
- *Zurich Airport* (Denoted as ICAO code: LSZH): The data was sourced from the [72], where the focus was on analyzing Zurich Airport flight arrivals.

2) DATASET PREPROCESSING

The dataset preprocessing involves data cleaning, transformation, resampling, smoothing, and scaling. We first omitted the preprocessing for incomplete trajectories. Then, we extracted positional states from the surveillance data, including latitude, longitude, and barometric altitude. Barometric altitude was chosen for its lower noise levels and greater reliability compared to GPS vertical accuracy [73]; moreover, it is commonly used for traffic control. The positional features were transformed into Cartesian vectors on the ENU (East-North-Up) coordinate centered at the airport. Then, trajectories were bounded by a radius, $r_{\max}$, determined by the farthest waypoints in the area chart of the Aeronautical Information Publication (AIP) with 120 km for Incheon airport, 100 km for Stockholm Arlanda airport, and 40 nautical miles for Zurich airport [72]. The trajectories were resampled to 1-second intervals without interpolating into a fixed number of timestamps. Then, the outlier states were removed, and the Savitzky-Golay filter [74] was applied to smooth the trajectories. Each trajectory was scaled to the range $[-1, 1]$ by dividing by r_{max}, such that $\mathcal{X}_i \leftarrow \mathcal{X}_i / r_{max}$. A preprocessed instance $\mathcal{X}_i$ has the states $x_{i,t} = \{x^x_{i,t}, x^y_{i,t}, x^z_{i,t}\}$. The trajectories are padded with NaN to $T_{\max} = \max_{i\in 1\ldots N}(T_i)$.

The datasets were split into training and testing sets with a 50/50 proportion, a common practice in time series datasets, as seen in the 128 UCR datasets [75] and 30 UEA datasets [76], where 50% training splits occur in 37 out of 158 datasets, and over 123 datasets use 50% or a smaller proportion for the training set. The small training proportion typically challenges the model's ability to generalize to unseen data but risks providing insufficient training data. We ensured a sufficient training set by including at least 8,000 training samples, which is relatively large compared to those in typical time series datasets found in the archives. Moreover, to establish reliable benchmarking, both classification and clustering share the same testing set, ensuring a consistent evaluation environment. As clustering

evaluation relies exclusively on the unseen testing set in the unsupervised setting, a significant amount of testing data should be allocated to guarantee the reliability and importance of clustering evaluation.

3) DATASET ANNOTATION

The lack of labeled datasets restricts the advancement in developing classification models, which require training with true labels as done by [30]. The studies cited in [3], [4], [5], [31], [32] utilized clustering to generate labels; however, this method is commonly practiced in the artificial intelligence field as pseudo-labeling. Moreover, many clustering studies omit the labels in their evaluations, focusing on the proximity of instances within clusters [10], [11], [15], [16], [47], [58]. Nevertheless, the primary concern remains the fidelity of class assignments, assessed using the true labels. Thus, it is essential to have data annotated by humans, as in [77], [78], to align with humans' semantic understanding of data.

Under the current ATM paradigm, our understanding of trajectories is based on maneuvering procedures; therefore, the datasets were manually annotated using information published in the AIPs. The annotation of arrival trajectories at Incheon Airport and Zurich Airport followed the same procedures. For arrival corridors labeling, performing K-means on the set of all $\{x^x_{i,1}, x^y_{i,1}\}$ for $i \in 1 \dots N$ provided a good initialization of segments before manual adjustments. The runways were labeled by manually drawing linear classification lines on $\{x^x_{i,T_i}, x^y_{i,T_i}\}$ for $i \in 1 \dots N$, to classify the touchdown points. A similar manual classification on initial approach fixes (IAFs) was performed to classify the approach procedures. As a result, classes are defined as combinations of runways, IAFs, and Standard Terminal Arrival Routes (STARs). For the Incheon departure dataset, labeling departure corridors mirrored that of arrivals but was implemented using the set $\{x^x_{i,T_i}, x^y_{i,T_i}\}$. In the AIP, the pairs of dependent runways comply with the same Standard Instrument Departure (SID); we labeled each $\{x^x_{i,1}, x^y_{i,1}\}$ as a dependent runway pair, using the same manual classification procedure. The labels for departure data denote the SIDs. The annotation of the trajectory data at Stockholm Arlanda Airport was more straightforward than others, as the surveillance data were given with the flight plan data [71]. The STARs and the situated runways are recorded; therefore, after minimal corrections and labeling with the initial approach fixes, the classes are assigned as the combinations of runways, IAFs, and STARs. Figure 6 provides a comprehensive visualization of the trajectory datasets, along with four representative example classes selected from each dataset. The figure illustrates the dataset annotations, where each class corresponds to a specific arrival or departure procedure and its situated runway, coded using aeronautical notation from the AIPs, and represents distinct trajectory patterns across classes.

Each set of class descriptions was then converted to integers denoted as $Y = \{y_1, y_2, y_3, \dots y_N\}$ for $y_i \in \mathbb{N}$. It is important to emphasize that these instance-level labels will not be used in representation learning via the self-supervised ATSCC framework or any baseline methods discussed in this paper; they are only for evaluation purposes.

B. EXPERIMENTS

1) HYPERPARAMETERS AND REPRODUCTION REMARKS

In a self-supervised setting, fine-tuning hyperparameters is challenging without access to labels, and incorporating labels in any manner would compromise the self-supervised nature of the training. Under such circumstances, observing the loss function does not directly reflect model performance, and the validation set has limited use, as it offers little meaningful insight without labels. Consequently, many prior works train self-supervised or unsupervised models using only training data and evaluate the results on testing data. Hyperparameters are either fixed [25] or adjusted within a very limited grid search, as seen in [22], [26]. Following this approach, our experiments are conducted by training the model in a self-supervised manner with a restricted hyperparameter grid search space.

Our model employs the CLM configuration from [66], featuring 12 layers with a model dimension of 768, feed-forward network dimensions of 3072 using Gaussian Error Linear Unit (GELU) activation, 12 attention heads with a 0.35 drop rate, and a binomial random masking probability set at 0.2. Following [22], [25], the dimension of the representation $z_{i,t}$ is set at 320. The batch size, B, is set at 16 for training with the AdamW optimizer, using a learning rate of 1×10^{-5} and a weight decay (λ) for regularization set at 1×10^{-5}. For unscaled ATM trajectory data, we recommend using the RDP threshold as $\epsilon \cdot r_{max}$, where r_{max} denotes the maximum control radius. However, since our data has been scaled to $[-1, 1]$, we have omitted the notation for r_{max}. We performed a grid search based on performance evaluation over the RDP threshold, $\epsilon \in \{0.0001, 0.001, 0.01, 0.1\}$ and the temperature in loss function $\tau \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0\}$. For ATSCC, the learned representation $z_{i,t}$ represents all its previous states; thus, the last-timestamp representation z_{i,T_i} is extracted as the instance-level representation for evaluation. To reduce the computational burden, the trajectory dataset was downsampled every 5 seconds without standardization. To ensure the consistency of our results, we trained and evaluated the model on each individual dataset separately, using 5 different seeds for each. For all training and evaluation, this paper used Python 3.11.8 and PyTorch 2.0.1 with CUDA Toolkit 11.8, running on an NVIDIA GeForce RTX 4090 GPU.

2) REPRODUCTION OF BASELINES

This paper discusses the ATM trajectory recognition tasks at the instance level, including classification and clustering. Consistent with the training and evaluation of ATSCC, we downsampled the trajectories at 5-second intervals and used identical geometric features across all baselines with $x_{i,t} = \{x^x_{i,t}, x^y_{i,t}, x^z_{i,t}, x^{u_x}_{i,t}, x^{u_y}_{i,t}, x^{u_z}_{i,t}, x^r_{i,t}, x^{\sin\theta}_{i,t}, x^{\cos\theta}_{i,t}\}$.

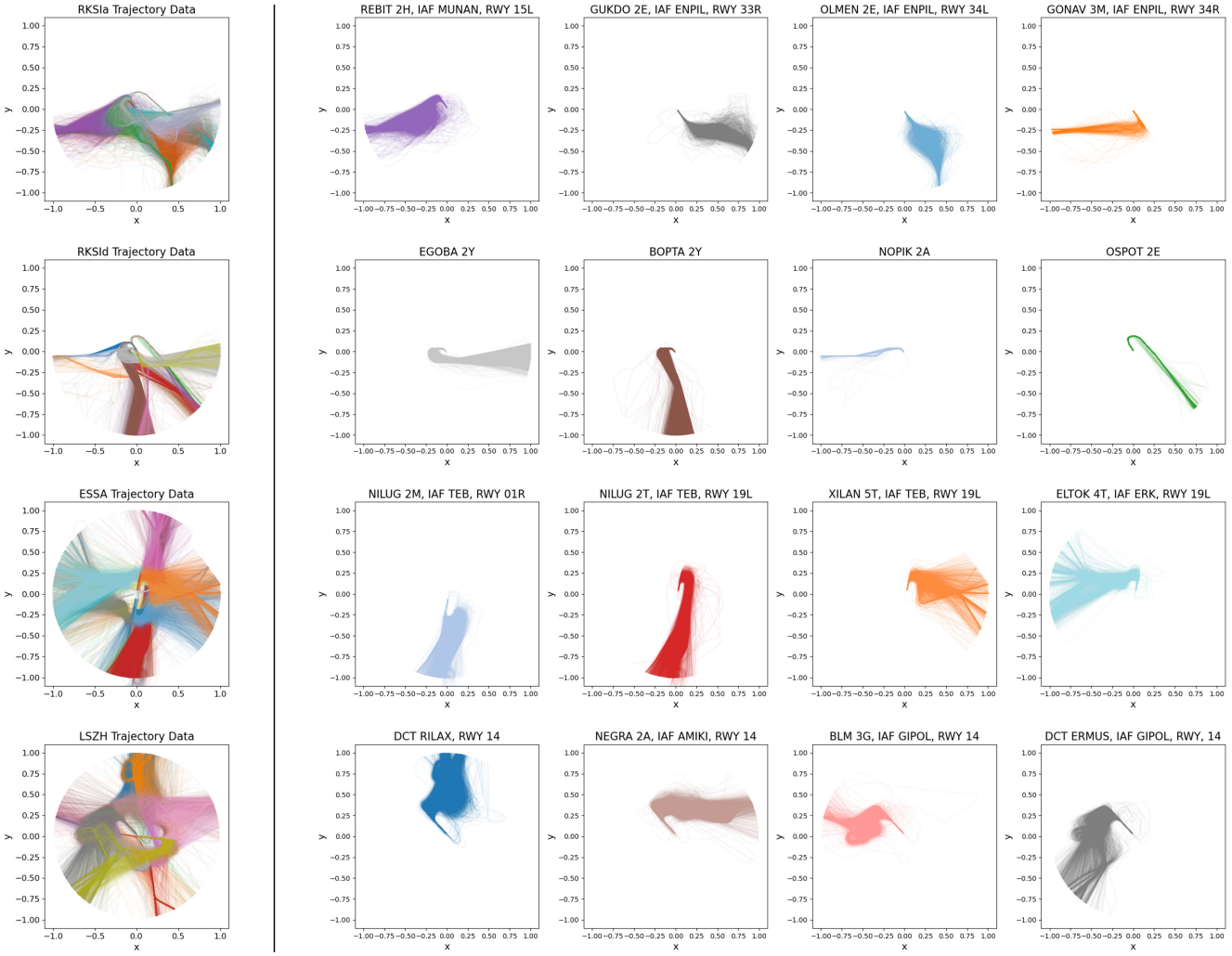


FIGURE 6. Visualization of the 2D top-view of preprocessed trajectory datasets and 4 example classes annotated by maneuvering procedures. In a dataset, each class corresponds to a specific arrival or departure procedure and its situated runway (In Order): Incheon Airport arrival and departure trajectories, Stockholm Arlanda Airport arrival trajectories, and Zurich Airport arrival trajectories.

Following [22], [25], the dimension of the representation vector is set to 320, except for TNC. We referred to the original code implementations, if publicly available, and the reproduction remarks provided by the authors of the baseline papers to reproduce the results. The reproduction details are described as follows:

- *SPIRAL* [9]: As an example of a non-neural network optimization-based method, we have reproduced SPIRAL using the DTW. We kept all hyperparameters default except for the representation vector size.
- *TCN-AE:* We use the Temporal Convolutional Network (TCN) architecture from [22] to construct this autoencoder. Following [18], [19], we employ the TCN as both the encoder and decoder with maxpooling and upsampling in the middle.
- *TF-AE:* We built a transformer autoencoder following the architecture in [20]. The encoding involves extracting features into a class token, while the decoding uses positional encoding as a time query on the latent vector for reconstruction, similar to [21].
- *T-Loss* [22] encourages the consistency of random subsequences using triplet loss. We used the default hyperparameters as those for the UCR and UEA datasets.
- *TNC* [23] leverages local smoothness of time series and uses a sliding window to encode the data into shorter sequences, which we use to represent the instances, as aggregation to a single vector was not demonstrated. We used the waveform CNN encoder and HAR configuration. The size of the sub-series representation was set to 64, with a window size of $\text{int}(T_{max}/20)$ and a sliding gap of 5.
- *TS-TCC* [24] maximize the agreement between two augmented samples. We referred to the hyperparameters for the HAR Dataset. Following [25], we exhibit the instance-level representation using Maxpooling on the base convolutional network.

TABLE 1. Performance of self-supervised representation learning baselines: Evaluation of learned representations across all trajectory datasets, measured by accuracy, NMI, and ARI scores.

	RKSIa			RKSId			ESSA			LSZH		
Baselines	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
Non-neural network												
• SPIRAL [9]	0.8344	0.6211	0.3102	0.9946	0.8732	0.7191	0.8503	0.7406	0.4352	0.9173	0.7920	0.6234
Autoencoders												
• TCN-AE	0.9753	0.6682	0.3593	0.9982	0.8706	0.6514	0.9942	0.7527	0.4420	0.9915	0.8060	0.6566
• TF-AE	0.9181	0.6420	0.2881	0.9971	0.8545	0.6229	0.9563	0.7489	0.4394	0.9674	0.7994	0.6159
Contrastive Learning												
• T-Loss [22]	0.9735	0.6660	0.3238	0.9984	0.8484	0.5969	0.9967	0.7552	0.4553	0.9916	0.8301	0.6349
• TNC [23]	0.9728	0.6982	0.4222	0.9980	0.8784	0.6771	0.9904	0.7581	0.4837	0.9897	0.8983	0.7908
• TS-TCC [24]	0.9847	0.7207	0.4304	0.9980	0.8831	0.6834	0.9989	0.7656	0.4670	0.9950	0.8667	0.7248
• TS2Vec [25]	0.9786	0.6890	0.3412	0.9984	0.8660	0.6691	0.9981	0.7572	0.4463	0.9933	0.8373	0.6381
• InfoTS [26]	0.9764	0.6668	0.3703	0.9986	0.8758	0.6621	0.9986	0.7623	0.4608	0.9930	0.8604	0.6851
• ATSCC	**0.9946**	**0.8723**	**0.8195**	**0.9987**	**0.9129**	**0.7723**	**0.9990**	**0.8640**	**0.6517**	**0.9977**	**0.9480**	**0.9037**

- *TS2Vec* [25] maximize contextual consistency of time series using the hierarchical contrastive loss. We used the reproduction details that the author demonstrated on the UCR and UEA datasets.
- *InfoTS* [26] implements a meta-learning technique in finding the best augmentation for representation learning. We reproduced it using the same hyperparameters as those for the UCR and UEA datasets.

3) TRAJECTORY CLASSIFICATION

An effective representation should maintain fidelity, preserving the semantic identity with respect to class labels, and enable better separability, meaning it distinguishes same-class representations from different-class ones. These properties can be evaluated using classification benchmarks, as instance-level labels Y were labeled based on AIPs and human interpretation of the data. The evaluation follows the same standard classification protocol as demonstrated in [22], [25]. This protocol involves training an SVM classifier with a Radial Basis Function (RBF) kernel on the learned representation of the training data, and evaluating the classification accuracy on the representation of the testing data. We intentionally use a simple classifier like SVM on top of the representation learning model to test the fidelity of the data representation in the feature space, encouraged by the encoder training procedure. With a complex classifier head like neural networks, it is difficult to determine whether improved results are due to the quality of the representation or the classifier itself [22]. For TNC, we fitted an SVM using the Global Alignment Kernel (GAK) [79]. We then assess the SVM's accuracy using the representations derived from the testing data. We have integrated the classification challenges in [3], [4], [30], [31], [32] by classifying both runways and approach patterns.

According to the accuracy results in Table 1, we observed a significant improvement in the accuracy of the arrival dataset for Incheon Airport, which features a complex configuration with four parallel runways. The experiment on departure trajectory data showed slightly better accuracy, as departure trajectories are relatively straightforward compared to arrivals. At Stockholm Arlanda Airport, which features two parallel runways, the results follow the same trend as those of Incheon arrivals, resulting in the highest accuracy on this largest dataset. Lastly, Zurich Airport has the most distinguishable pattern, yet we still observed significantly better performance compared to the other baselines.

The ATSCC model outperforms all baselines in accuracy, demonstrating that its learned representation is class-preserving and easily separable in the embedding space. This leads to significant improvements in the classifier's ability to identify combinations of runways and terminal maneuvering procedures, outperforming state-of-the-art baselines and highlighting the suitability of ATSCC for air traffic trajectory data. We visualized the learned representations of each model in the embedding space by transforming them into two dimensions using t-SNE, as shown in Figure 7, to comparatively highlight the outstanding ability of ATSCC in generating effective representations with strong class separability, where instances from distinct classes form well-separated clusters.

4) TRAJECTORY CLUSTERING

We define uniformity as the degree to which only same-class representations are closely grouped in the embedding space. We evaluate this property to assess whether a simple clustering algorithm, when employed on the learned representation, can effectively differentiate data instances and ensure correct clustering without relying on true labels.

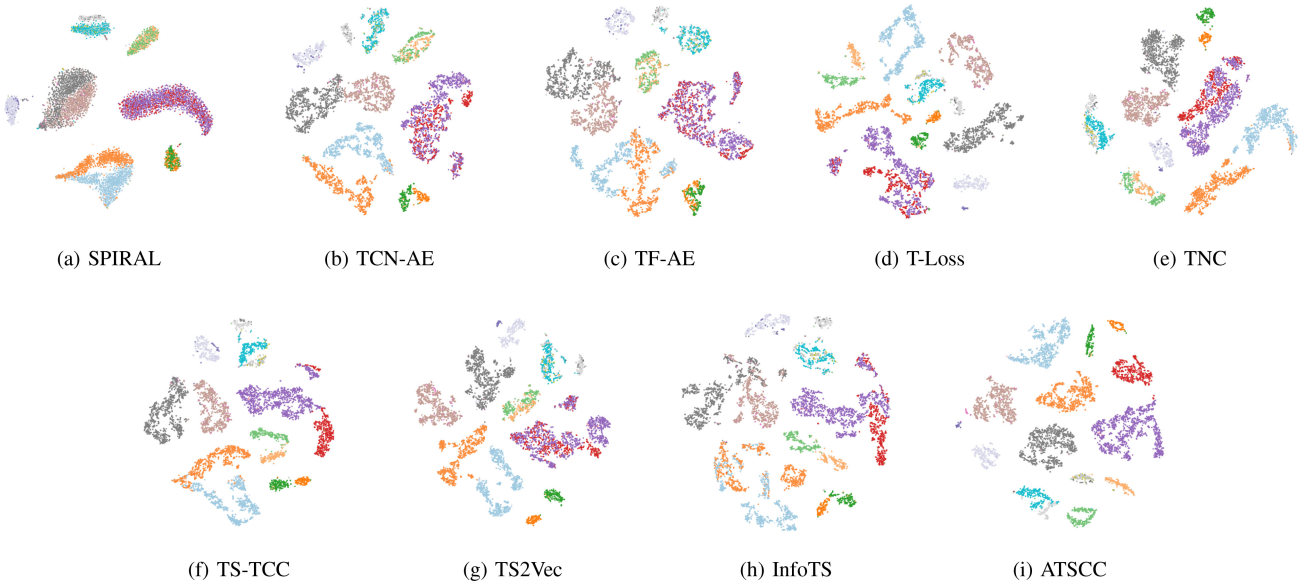


FIGURE 7. t-SNE visualization of learned representations from the arrival trajectory dataset at Incheon International Airport (RKSIa) across all representation learning baselines. The learned representations obtained from ATSCC (i) demonstrate outstanding class separation, with instances from each class forming clearly grouped clusters.

We used K-Means for benchmarking, as it is particularly suitable for this purpose due to its distance-based nature, which is directly influenced by how well-distributed the encoded features are. Thus, we conduct the clustering experiment similarly to the benchmarking protocol in [23] by implementing K-Means on the instance-level representation of trajectories using Euclidean distance, and setting the number of clusters equal to the number of unique test labels; however, for TNC, DTW distance was used.

Due to the absence of instance labels in recent ATM clustering works (e.g., [12], [34], [36], [56], [57]), clusters are typically analyzed visually. Besides, metrics such as the Silhouette score, Calinski-Harabasz Index, and Davies-Bouldin Index (DBI) were used (e.g., [10], [11], [15], [16], [47], [58]). However, the fidelity and accuracy of cluster assignments are more crucial than the proximity of representations within clusters. Therefore, following recent clustering literature [60], [80], [81], [82], we use the Normalized Mutual Information (NMI) score and Adjusted Rand Index (ARI) to assess the similarity between the clustering results and the ground truth labels, where NMI measures label preservation and ARI evaluates pairwise sample accuracy. This evaluation method provides a sensible performance assessment by testing whether the learned representation enhances clustering to recognize unique maneuvering procedures and correctly assign labels to the samples.

However, NMI and ARI have some limitations [83], which are addressed in our experiment. ARI requires true class labels, and NMI assumes a one-to-one correspondence between clusters and classes. These conditions hold in our case, as the ground-truth labels are provided, and each class corresponds to a distinct maneuvering procedure. While ARI is sensitive to imbalanced cluster sizes and assumes random cluster assignments, these issues are mitigated by using k-means clustering, which minimizes intra-cluster variance and typically produces balanced clusters. Additionally, as these metrics do not provide insight into the cluster structure, we address this issue by complementing them with t-SNE visualizations (Figure 7) to gain a better understanding of the cluster structure.

The NMI and ARI results, tabulated in Table 1, show that both scores are significantly enhanced across arrival datasets, ranging from those with complex arrival trajectories, such as Incheon and Stockholm Arlanda with their parallel runways, to those with less ambiguous configurations, like Zurich. Although the other baselines perform well with departure data, our approach still achieves superior scores. These results validate that the learned representation aligns well with the operational semantics. For example, trajectories with landing patterns that are geometrically similar become distinguishable in the embedding space. The superior scores reflect the framework's capability to generate the representations with exceptional uniformity, enabling K-Means to establish high-fidelity clusters. The experiment further confirms that, compared to the baselines, the ATSCC framework is the most suitable representation learning method for air traffic management trajectory data. Moreover, besides separability, uniformity can also be observed in Figure 7, and ATSCC, in particular, produces the most clearly defined and groupable clusters.

We further conducted a clustering experiment using a greater number of clusters than the number of classes, as trajectory clustering is typically employed for trajectory categorization, which requires more clusters than procedures, according to [11], [32], [47]. This experiment aimed to assess the ability of the representation to enable clustering to

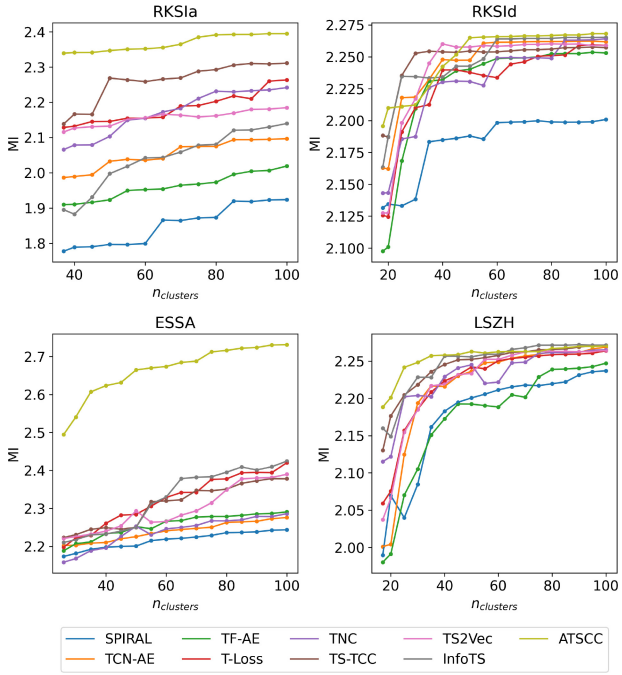


FIGURE 8. Comparison of MI score plot evaluation on learned representation using K-means clustering: Number of clusters ranges from the number of classes Up to 100, incremented in steps of 5.

generate high-fidelity subclusters, as ideally, these clusters should be subclasses of the procedures. To maximize fidelity, it is important to maximize mutual information [26], [27]. Therefore, we evaluate the Mutual Information (MI) score, which shares the same limitations as the NMI score, on clusters generated by k-means, with the number of clusters ranging from the number of existing classes to 100.

Figure 8 illustrates that our learned representation outperforms other methods for complex airports like Incheon and Stockholm Arlanda. However, for simpler cases such as Zurich and the departure trajectory of Incheon airports, ATSCC demonstrates slightly better performance due to minimal runway ambiguity. The MI scores converge to higher values, highlighting the importance of representation learning over the choice of clustering method and parameters. The results demonstrate that increasing the number of clusters to separate ambiguous groups is ineffective without an effective data representation. Along with classification and clustering benchmarks, this further highlights that the separability and uniformity of clusters lead to higher fidelity and accuracy in downstream tasks.

5) EFFECTIVENESS OF THE LEARNED REPRESENTATION

The results highlight the importance of learned representations in enhancing downstream classification and clustering. As shown in Figure 7, ATSCC distinctly groups same-class samples without ambiguity or mixing of other-class samples in the embedding space after encouraging consistency within segments. As shown in Table 1 and Figure 8, our method significantly outperforms others, particularly for arrival trajectories at airports with complex parallel runway configurations, where traffic patterns become complex. Traditional methods, such as non-neural network approaches and autoencoders, perform poorly by relying on raw trajectory distances, treating all flight phases equally without recognizing operational distinctions. This biases them toward distant states while underemphasizing closer, operationally distinct paths, such as the landing phase. Consequently, their representations remain ambiguous, especially for clusters representing the same arrival procedure but differing in landing runways. This issue extends to departure trajectories, though with less complexity than arrival trajectories.

While various contrastive learning approaches have been successful in many types of time series data, they underperform on air traffic trajectory data. T-Loss [22] and TS2Vec [25] align an incomplete trajectory with a more advanced one due to missing information. For instance, in arrival cases, this may result in the model generalizing trajectories landing on different runways as the same by aligning their representations with the representation of their identical incomplete sub-trajectories. TNC [23] may treat different flight phases as similar due to false flight phase grouping. For example, if two trajectories share an approach path but lead to different runways, the model may align their landing paths through the shared approach path. Additionally, since TNC infers only sub-trajectory-level representation, it suffers from higher dimensionality compared to other methods. TS-TCC [24] experiences procedural distortion, and its augmentations do not sufficiently promote separation and uniformity in the embedding space. The model learns distorted procedural characteristics from augmented positive pairs, leading to deviations from the original procedure and less effective grouping of same-class trajectories. InfoTS [26] combines augmentations with meta-learning, but its augmentation methods are unsuitable for air traffic trajectories, resulting in either missing information or procedural distortion.

ATSCC, on the other hand, avoids randomly cropping important information, preserves procedural details, and draws clear boundaries between flight phases. It ensures similar representation for sub-trajectories with the same procedure while separating those with different procedures. When tested on complete trajectories, ATSCC ensures class separability and uniformity in the embedding space, enabling downstream algorithms to effectively classify and cluster the samples, resulting in superior benchmarking scores.

C. ANALYSIS

1) ABLATION STUDY

In this section, we evaluate the effectiveness of the framework's components by evaluating the average performance on four trajectory datasets upon modifications, as tabulated in Table 2. We begin by analyzing the geometric feature extraction by excluding certain features and examining the impact on the results. Polar positions have a greater

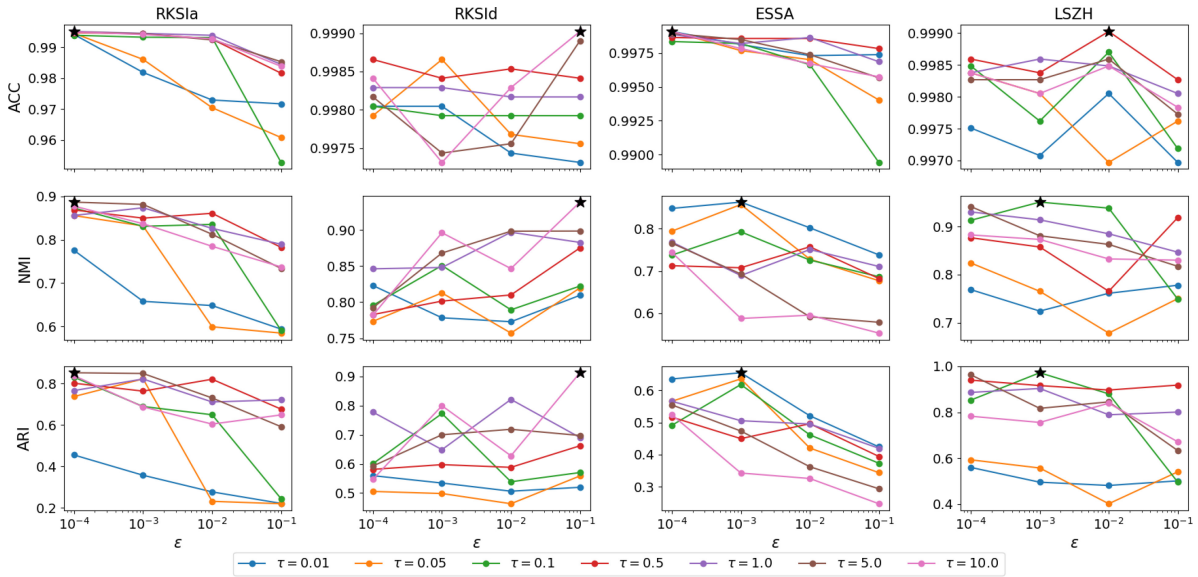


FIGURE 9. Sensitivity analysis of model performance across RDP epsilon (ϵ) and temperature (τ): Experimentation on four aircraft trajectory datasets, evaluated using Accuracy, NMI, and ARI. The optimal combinations of ϵ and τ are highlighted in black.

TABLE 2. Ablation results on four trajectory datasets.

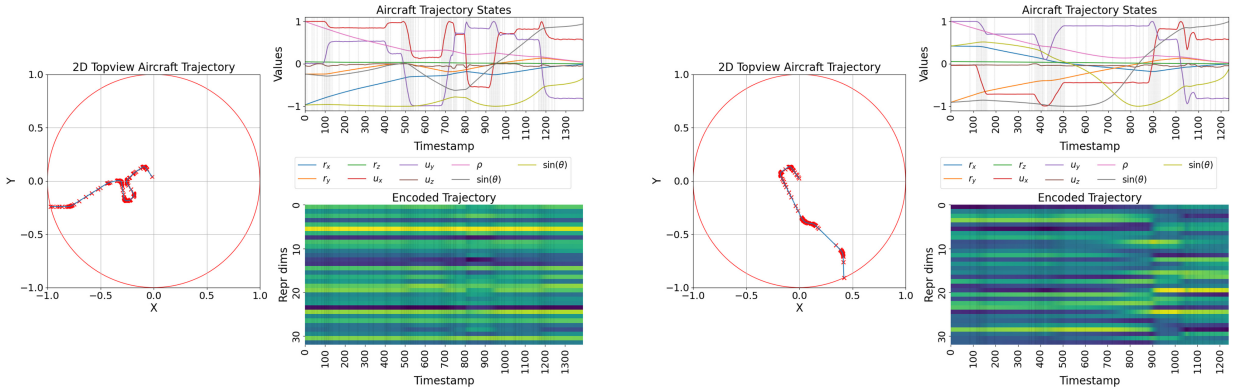
Configurations	ACC	NMI	ARI
Full Configuration	0.9977	0.8993	0.7868
Geometric Feature Extraction			
• without aircraft path vectors	0.9971	0.8751	0.7537
• without polar positions	0.9966	0.7883	0.5624
• catesian positions only	0.9974	0.8080	0.6588
Random Masking			
• without random masking	0.9962	0.8044	0.5818
L2 Normalization			
• without token L2 norm	0.9977	0.8941	0.7691
• without representation L2 norm	0.9952	0.7902	0.5784
• without both L2 norm	0.9961	0.8090	0.6089
Positional Encoding			
• Sinusoidal	0.9723	0.1863	0.0712
• Learnable	0.9892	0.7751	0.6400
Backbone Architecture			
• LSTM	0.6195	0.5359	0.2606
• Dilated Causal Convolution	0.9456	0.4903	0.2752
Soft-nearest Neighbor Loss			
• $\mathcal{L}_{snn}$: Equation (7)	0.9973	0.8718	0.7361

impact on performance than path vector sequences, as they change rapidly near the airport, resulting in distinct tokens. Conversely, adding path sequences as the only additional feature reduces scores due to potential confusion from parallel landing paths. However, we found that including both polar positions and path sequence significantly enhances overall performance.

We observed that applying random binomial masking on the input and attention masks significantly contributes to the effectiveness of the representations, as removing the binomial masking leads to a noticeable drop in performance. This is because the random masking enhances regularization and generalization, performing similarly to dropout. Additionally, it introduces data variation similar to weak augmentation. Random timestamp masking in each training epoch simulates inconsistent transmissions, internally generating new sample variations and empirically enhancing the model's ability to produce effective trajectory representations.

L2 normalization, which constrains the representation within a hypersphere, can enhance downstream task performances by promoting linear separation in the embedding space. Therefore, we observed that eliminating L2 normalization after both projection layers resulted in significant performance loss, similar to L2 normalizing only on tokens. On the other hand, L2 normalization on the representation significantly enhanced the scores, consistent with [67]; however, a slight additional improvement was observed when adding token normalization.

Since the ATSCC encoder does not incorporate explicit positional encoding, we compared our results with those obtained using commonly employed positional encodings. We referenced [20] for sinusoidal positional encoding and [84] for the initialization of the learnable positional encoding. We observed that the encoder following the NoPos [66] outperforms those with explicit positional encoding, which can alter the information in tokens. For self-supervised learning, it is crucial to utilize the underlying data information, rather than relying solely on the expected output, as in supervised learning. Introducing



(a) Trajectory of Class 8: Arriving via STAR REBIT2, Landing on Runway 15L via IAF MUNAN

(b) Trajectory of Class 28: Arriving via STAR OLMEN2H, Landing on Runway 15L via IAF MUNAN

FIGURE 10. Visualization of embedded trajectories: 2D top-view trajectory plot and geometric features.

distortions through positional encoding can fundamentally impact training.

We verify the suitability of the causal transformer by implementing different encoder backbones. The ATSCC performs binomial masking on both the input and the attention matrices. However, different architectures perform this random masking differently. The dilated causal convolution network encoder architecture and its random masking technique are referenced in [25]. Later, we replace the backbone of this CNN model with a 3-layer LSTM model having a hidden layer dimension of 512. Both encoders lead to a significant decrease in overall performance, as CNN can become locally focused, and LSTM loses information when encoding long sequences.

The soft-nearest neighbor loss [68] was rearranged and modified into Equation (8). We have empirically demonstrated that omitting positive contrasting terms in the sum with negatives yields better performance in our framework and experimental setting.

2) SENSITVITY ANALYSIS

Since ATSCC incorporates segment ID assignment (Algorithm 1) into the SNNL (Equation (8)), for each dataset, we conducted a grid search for two critical hyperparameters: the RDP threshold (ϵ) and the loss temperature (τ). The grid search was performed during the initial training with a random seed, and the training was then repeated with four additional seeds, using the same hyperparameters. Figure 9 shows the grid search results for these hyperparameters.

The threshold ϵ determines segment granularity, with smaller values capturing finer details but introducing numerous small segments, while larger values risk overlooking critical information. Increasing ϵ tends to reduce performance in airports with complex configurations, as capturing detailed traffic patterns, particularly in arrival datasets, becomes more crucial. Conversely, the learned representation for less complex procedures, such as those in departure datasets, can perform better with higher threshold

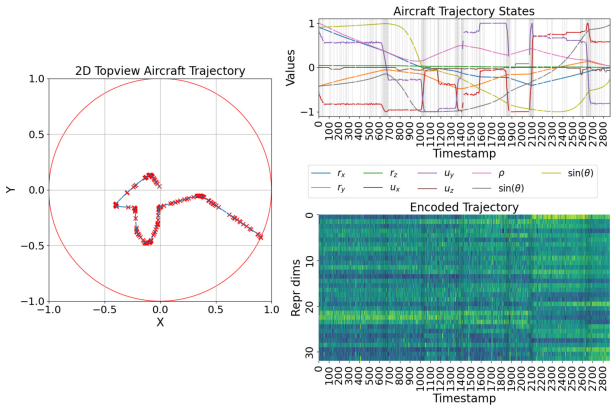


FIGURE 11. Visualization of an embedded low-quality trajectory, 2D top-view trajectory plot and geometric features.

values, as departure procedures are generally simpler than arrival procedures.

The temperature τ controls the sharpness of the loss, where lower values enforce stricter grouping, and higher values promote greater sparsity in the embedding space. In many cases, the learned representation generally performs acceptably when the model is trained with $\tau = 1$. Therefore, starting with $\tau = 1$ is recommended, followed by adjusting the loss temperature to explore potential performance improvements. The performance of the learned representation is maximized with the optimal combination of ϵ and τ.

3) VISUAL EXPLANATION OF EMBEDDING

For ease of visual explanation, we have selected two trajectories from the Incheon arrival dataset and their learned representations at all timestamps, which were trained with an output dimension of 32. Both flights landed on the same runway but arrived via different STARs. At each timestamp, trajectory states in different scales and units are collectively converted into scale-invariant representation vectors. According to Figure 10, the representation $z_{i,t}$

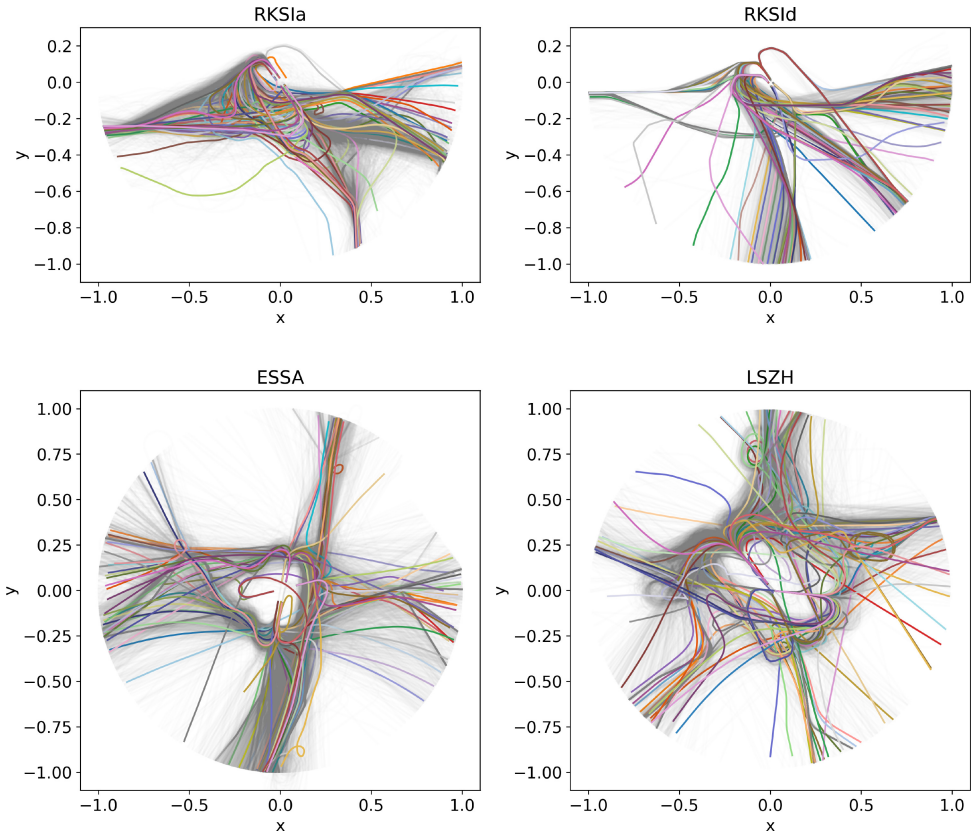


FIGURE 12. Visualization of clustering results on four trajectory datasets: Illustrating representative trajectories using the mean of cluster members.

changes over time as causal attention collects information $x_{i,t}$ in $\mathcal{X}'_{i,t} \subseteq \mathcal{X}_i$ from current and previous states at each timestamp $t \in \{1, \ldots, t\}$. As we enforce consistency within segments, $z_{i,t}$ remains consistent within each segment, as clearly visualized. A clear difference in $z_{i,t}$ of the two trajectories is noticeable at earlier timestamps due to different STARs. Later, $z_{i,t}$ of two trajectories diverge, despite landing on the same runway, because the information of the STARs was carried along while encoding. The ATSCC does not forcibly discretize the states, nor does it use the notation of Euclidean vectors or waypoints. Instead, the values $z_{i,t}$ represent scale-invariant semantic embeddings that preserve temporal transitions in continuous embedding space. This enables their applicability to machine learning models, which can be flexibly adapted for other tasks.

4) ROBUSTNESS TO LOW-QUALITY DATA

This section examines the model's robustness to sparse and low-quality data, particularly in scenarios where inconsistent ADS-B broadcasts lead to missing transmissions and gaps in trajectory states. To simulate this, we randomly remove 10% of the states in each instance $\mathcal{X}_i$ from the test dataset and encode them using an encoder trained on the original high-quality training data. This scenario represents a model trained on high-quality data but deployed in an environment with low-quality data. From an example shown in Figure 11, although noisy $z_{i,t}$ values are present, the segmentation remains observable, with all $z_{i,t}$ consistently similar within $\{z_{i,t} \mid \upsilon_{i,t} = \upsilon\}$ for all $\upsilon \in \text{unique}(\Upsilon_i)$, as the encoder can refer to only existing states. As shown in Table 3, ATSCC exhibits only a slight decrease in accuracy, NMI, and ARI when tested on low-quality datasets, thereby maintaining robust performance. Under this data condition, its scores still surpass the average performance of all baseline methods, evaluated on the original testing set, shown in Table 1. ATSCC representations can still retain fidelity to class labels while ensuring separability and uniformity in the embedding space, demonstrating its robustness to sparse and low-quality data, a common scenario in real-world operations.

TABLE 3. Average model performance on low-quality data.

Data Condition	ACC	NMI	ARI
Original Test Data	0.9977	0.8993	0.7868
Low-quality Test Data	0.9953	0.8488	0.7273

5) ANALYSIS FOR TRAJECTORY CATEGORIZATION

Many previous studies have focused on air traffic categorization; therefore, this section shows the feasibility of our model in addressing this problem. The trajectory can be analyzed by varying the number of clusters to 100 or more, as suggested by [13], [32]. Figure 8 shows that increasing the number of clusters maximizes the MI score; thus, we

performed a brief analysis using 100 clusters as an example of clustering results for a categorization task.

Since the trajectory data undergoes semantic analysis using the learned representation, trajectories with spatially close but operationally distinct segments, such as parallel landing paths, can be differentiated, as demonstrated by the results on Incheon and Stockholm Arlanda Airports (Figure 12). Unlike other studies that address complex airports with parallel runways (e.g., [3], [4], [10], [11], [32], [38], [58]), ATSCC effectively enables the separation of ambiguous classes without requiring airport information, unlike [47], resulting in high-fidelity subclasses of the procedures. Similar to [13], the visualization highlights the identification of non-standard patterns, notably at Incheon Airport, and trajectories with holding patterns are clearly grouped at both Stockholm Arlanda and Zurich Airports. This confirms that the ATSCC is also applicable for characterizing trajectories and outlier detection.

V. CONCLUSION

This paper has proposed a representation learning framework for multivariate air traffic trajectory data. ATSCC trains a causal transformer encoder by assigning segment IDs via the RDP algorithm for training with the soft-nearest neighbor loss. We proposed fidelity benchmarking using manually labeled trajectory data, guided by AIPs, to evaluate the representation. The classification and clustering results indicate that ATSCC is the most suitable framework for ATM trajectory data and is robust for low-quality data. Our learned representation enables the separation of ambiguous paths that are semantically different, non-standard patterns, and holding points. We have addressed the limitations of several prior works, for ATSCC does not rely on computationally intensive distance metrics, nor does it require human-specified aeronautical information. Patterns are recognized automatically through the training, showing adaptability to various airport configurations. The recommendations for future work include extending the model's applicability to other trajectory or time series data, as well as adapting the model to tasks such as trajectory prediction, imputation, or anomaly detection. Moreover, ATSCC can potentially train a pre-trained trajectory model for various machine-learning tasks. Additionally, incorporating weather or operational data could extend the model's utility across diverse scenarios.

REFERENCES

[1] G. Enea and M. Porretta, "A comparison of 4D-trajectory operations envisioned for nextgen and sesar, some preliminary findings," in *Proc. 28th Congr. Int. Council Aeronaut. Sci.*, Sep. 2012, pp. 23–28.

[2] M. López-Lago, J. Serna, R. Casado, and A. Bermúdez, "Present and future of air navigation: PBN operations and supporting technologies," *Int. J. Aeronaut. Space Sci.*, vol. 21, no. 2, pp. 451–468, Oct. 2019.

[3] M. C. R. Murça, R. DeLaura, R. J. Hansman, R. Jordan, T. Reynolds, and H. Balakrishnan, "Trajectory clustering and classification for characterization of air traffic flows," in *Proc. 16th AIAA Aviat. Technol., Integr., Oper. Conf.*, Jun. 2016, p. 3760.

[4] M. C. R. Murça, R. J. Hansman, L. Li, and P. Ren, "Flight trajectory data analytics for characterization of air traffic flows: A comparative analysis of terminal area operations between New York, Hong Kong and Sao Paulo," *Transp. Res. Part-C, Emerg. Technol.*, vol. 97, pp. 324–347, Dec. 2018.

[5] S. Madar, T. G. Puranik, and D. N. Mavris, "Application of trajectory clustering for aircraft conflict detection," in *Proc. IEEE/AIAA 40th Digit. Avionics Syst. Conf. (DASC)*, Oct. 2021, pp. 1–9.

[6] F. Rehm, "Clustering of flight tracks," in *Proc. AIAA Infotech@Aerospace*, Apr. 2010, p. 3412.

[7] L. Wu, I. E.-H. Yen, J. Yi, F. Xu, Q. Lei, and M. Witbrock, "Random warping series: A random features method for time-series embedding," in *Proc. 21st Int. Conf. Artif. Intell. Statist.*, Apr. 2018, pp. 793–802.

[8] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 26, no. 1, pp. 43–49, Feb. 1978.

[9] Q. Lei, J. Yi, R. Vaculin, L. Wu, and I. S. Dhillon, "Similarity preserving representation learning for time series clustering," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2845–2851.

[10] W. Zeng, Z. Xu, Z. Cai, X. Chu, and X. Lu, "Aircraft trajectory clustering in terminal airspace based on deep autoencoder and Gaussian mixture model," *Aerospace*, vol. 8, no. 9, p. 266, 2021.

[11] X. Chu, X. Tan, and W. Zeng, "A clustering ensemble method of aircraft trajectory based on the similarity matrix," *Aerospace*, vol. 9, no. 5, p. 269, 2022.

[12] X. Olive, L. Basora, B. Viry, and R. Alligier, "Deep trajectory clustering with autoencoders," in *Proc. 9th Int. Conf. Res. Air Transp. ICRAT*, 2020, pp. 1–8.

[13] Y. Liu, K. K. H. Ng, N. Chu, K. K. Hon, and X. Zhang, "Spatiotemporal image-based flight trajectory clustering model with deep convolutional autoencoder network," *J. Aerosp. Inf. Syst.*, vol. 20, no. 9, pp. 575–587, Sep. 2023.

[14] P. Malhotra, T. V. Vishnu, L. Vig, P. Agarwal, and G. Shroff, "TimeNet: Pre-trained deep recurrent neural network for time series classification," 2017, *arXiv:1706.08838*.

[15] G. Liu, Y. Fan, J. Zhang, P. Wen, Z. Lyu, and X. Yuan, "Deep flight track clustering based on spatial–temporal distance and denoising auto-encoding," *Expert Syst. Appl.*, vol. 198, Jul. 2022, Art. no. 116733.

[16] Y. Fan, J. Liu, H. Ye, and Z. Lyu, "TA-LSTM: A time and attribute aware LSTM for deep flight track clustering," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 5, pp. 7047–7060, Oct. 2023.

[17] A. van den Oord et al., "WaveNet: A generative model for raw audio," 2016, *arXiv:1609.03499*.

[18] R. Mo et al., "Unsupervised TCN-AE-based outlier detection for time series with seasonality and trend for cellular networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3114–3127, May 2023.

[19] M. Thill, W. Konen, H. Wang, and T. Bäck, "Temporal convolutional autoencoder for unsupervised anomaly detection in time series," *Appl. Soft Comput.*, vol. 112, Nov. 2021, Art. no. 107751.

[20] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. I. Guyon et al., Eds., Red Hook, NY, USA: Curran Assoc., Inc., 2017.

[21] G. Tevet, B. Gordon, A. Hertz, A. H. Bermano, and D. Cohen-Or, "Motionclip: Exposing human motion generation to clip space," in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV)*, Tel Aviv, Israel, 2022, pp. 358–374.

[22] J.-Y. Franceschi, A. Dieuleveut, and M. Jaggi, "Unsupervised scalable representation learning for multivariate time series," in *Advances in Neural Information Processing Systems*, vol. 32. H. Wallach, H. Larochelle, A. Beygelzimer, F. d' Alché-Buc, E. Fox, and R. Garnett, Eds., Red Hook, NY, USA: Curran Assoc., Inc., 2019.

[23] S. Tonekaboni, D. Eytan, and A. Goldenberg, "Unsupervised representation learning for time series with temporal neighborhood coding," in *Proc. Int. Conf. Learn. Represent.*, Sep. 2020, pp. 1–17.

[24] E. Eldele et al., "Time-series representation learning via temporal and contextual contrasting," in *Proc. 13th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 2352–2359.

[25] Z. Yue et al., "TS2Vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8980–8987.

[26] D. Luo et al., "Time series contrastive learning with information-aware augmentations," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 4534–4542.

[27] Y. Tian, C. Sun, B. Poole, D. Krishnan, C. Schmid, and P. Isola, "What makes for good views for contrastive learning?" in *Proc. 34th Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6827–6839.

[28] Z. Islam, M. Abdel-Aty, and B. M. T. H. Anik, "Transformer-conformer ensemble for crash prediction using connected vehicle trajectory data," *IEEE Open J. Intell. Transp. Syst.*, vol. 4, pp. 979–988, 2023.

[29] J. Ugan, M. Abdel-Aty, and Z. Islam, "Using connected vehicle trajectory data to evaluate the effects of speeding," *IEEE Open J. Intell. Transp. Syst.*, vol. 5, pp. 16–28, 2024.

[30] C. S. Bosson and T. Nikoleris, "Supervised learning applied to air traffic trajectory classification," in *Proc. AIAA Inf. Syst.-AIAA Infotech @ Aerosp.*, Jan. 2018, pp. 1–22.

[31] N. Jiménez-Campfens, A. Colomer, J. Núñez, J. M. Mogollón, A. L. Rodríguez, and V. Naranjo, "Deep learning in aeronautics: Air traffic trajectory classification based on weather reports," in *Proc. 21st Int. Conf. Intell. Data Eng. Autom. Learn. (IDEAL)*, Guimaraes, Portugal, 2020, pp. 148–155.

[32] C. Deng, H.-C. Choi, H. Park, and I. Hwang, "Trajectory pattern identification and classification for real-time air traffic applications in area navigation terminal airspace," *Transp. Res. Part-C, Emerg. Technol.*, vol. 142, Sep. 2022, Art. no. 103765.

[33] D. I. Tselentis and E. Papadimitriou, "Driver profile and driving pattern recognition for road safety assessment: Main challenges and future directions," *IEEE Open J. Intell. Transp. Syst.*, vol. 4, pp. 83–100, 2023.

[34] L. Basora, J. Morio, and C. Mailhot, "A trajectory clustering framework to analyse air traffic flows," in *Proc. 7th SESAR Innovat. Days*, Nov. 2017, pp. 1–8.

[35] J. Abidi and F. Filali, "Mobility analytics of fans during the 2021 FIFA Arab cup™ football tournament in Qatar," *IEEE Open J. Intell. Transp. Syst.*, vol. 4, pp. 653–662, 2023.

[36] S. J. Corrado, T. G. Puranik, O. J. Pinon, and D. N. Mavris, "Trajectory clustering within the terminal airspace utilizing a weighted distance function," *Proceedings*, vol. 59, no. 1, p. 7, 2020.

[37] D. Müllner, "Modern hierarchical, agglomerative clustering algorithms," 2011, *arXiv:1109.2378*.

[38] G. Andrienko, N. Andrienko, P. Bak, D. Keim, and S. Wrobel, *Visual Analytics of Movement*. Berlin, Germany: Springer, 2013.

[39] S. Rinzivillo, D. Pedreschi, M. Nanni, F. Giannotti, N. Andrienko, and G. Andrienko, "Visually driven analysis of movement data by progressive clustering," *Inf. Vis.*, vol. 7, nos. 3–4, pp. 225–239, Sep. 2008.

[40] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "Optics: Ordering points to identify the clustering structure," *SIGMOD Rec.*, vol. 28, no. 2, pp. 49–60, Jun. 1999.

[41] G. Andrienko, N. Andrienko, G. Fuchs, and J. M. C. Garcia, "Clustering trajectories by relevant parts for air traffic analysis," *IEEE Trans. Vis. Comput. Graph.*, vol. 24, no. 1, pp. 34–44, Jan. 2018.

[42] P. C. Besse, B. Guillouet, J.-M. Loubes, and F. Royer, "Review and perspective for distance-based clustering of vehicle trajectories," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 11, pp. 3306–3317, Nov. 2016.

[43] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. Int. Pac.-Asia Conf. Knowl. Discovery Data Min.*, 2013, pp. 160–172.

[44] U. Ramer, "An iterative procedure for the polygonal approximation of plane curves," *Comput. Graph. Image Process.*, vol. 1, no. 3, pp. 244–256, Nov. 1972.

[45] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica, Int. J. Geograph. Inf. Geovis.*, vol. 10, no. 2, pp. 112–122, Dec. 1973.

[46] L. Rdusseeun and P. Kaufman, "Clustering by means of medoids," in *Proc. Statist. Data Anal. Based L1 Norm Conf.*, Neuchatel, Switzerland, vol. 31, 1987, pp. 1–12.

[47] C. Yin, W. Zeng, H. Jiang, X. Tan, and W. Tian, "Standard procedure-guided flight trajectory pattern mining for airport terminal airspace," *Int. J. Aeronaut. Space Sci.*, vol. 25, no. 4, pp. 1412–1427, May 2024.

[48] G. Xuhao, Z. Junfeng, and P. Zihan, "Trajectory clustering for arrival aircraft via new trajectory representation," *J. Syst. Eng. Electron.*, vol. 32, no. 2, pp. 473–486, 2021.

[49] A. Mcfadyen, M. O'Flynn, T. Martin, and D. Campbell, "Aircraft trajectory clustering techniques using circular statistics," in *Proc. IEEE Aerosp. Conf.*, 2016, pp. 1–10.

[50] A. Bhattacharyya, "On a measure of divergence between two multinomial populations," *Sankhyā: Indian J. Statist. (1933-1960)*, vol. 7, no. 4, pp. 401–406, 1946.

[51] J.-G. Lee, J. Han, and K.-Y. Whang, "Trajectory clustering: A partition-and-group framework," in *Proc. ACM SIGMOD Int. Conf. Manag. Data*, 2007, pp. 593–604.

[52] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, 1996, pp. 226–231.

[53] M. Gariel, A. N. Srivastava, and E. Feron, "Trajectory clustering and an application to airspace monitoring," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 4, pp. 1511–1524, Dec. 2011.

[54] X. Olive and J. Morio, "Trajectory clustering of air traffic flows around airports," *Aerosp. Sci. Technol.*, vol. 84, pp. 776–781, Jan. 2019.

[55] J. MacQueen et al., "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probabil.*, Oakland, CA, USA, 1967, pp. 281–297.

[56] S. Chakrabarti and A. E. Vela, "Modeling and characterizing aircraft trajectories near airports using extracted control actions," *J. Aerosp. Inf. Syst.*, vol. 20, no. 2, pp. 81–101, 2023.

[57] W. J. Eerland and S. Box, "Trajectory clustering, modeling and selection with the focus on airspace protection," in *Proc. AIAA Infotech @ Aerosp.*, Jan. 2016, pp. 1–14.

[58] J. J. Damanik, C. Shin, and H.-L. Choi, "Spatial trajectory clustering of incheon airport flights," in *Proc. Korean Soc. Aeronaut. Space Sci. Fall Conf.*, Nov. 2022, pp. 284–285.

[59] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. Roy. Statist. Soc. Ser. B, Statist. Methodol.*, vol. 39, no. 1, pp. 1–22, Sep. 1977.

[60] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 478–487.

[61] M. Chen, Z. Xu, A. Zeng, and Q. Xu, "Fraug: Frequency domain augmentation for time series forecasting," 2023, *arXiv:2302.09292*.

[62] Y. Lee, Y. Byun, and J.-G. Baek, "Time series anomaly detection using contrastive learning based one-class classification," in *Proc. Int. Conf. Artif. Intell. Inf. Commun. (ICAIIC)*, 2023, pp. 330–335.

[63] H. Fabian. "RDP." 2016. [Online]. Available: https://github.com/fhirschmann/rdp/

[64] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," in *Proc. OpenAI*, 2019, pp. 1–24.

[65] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., Red Hook, NY, USA: Curran Assoc., Inc., 2020, pp. 1877–1901.

[66] A. Haviv, O. Ram, O. Press, P. Izsak, and O. Levy, "Transformer language models without positional encodings still learn positional information," in *Proc. Find. Assoc. Comput. Linguist. (EMNLP)*, Dec. 2022, pp. 1382–1390.

[67] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9929–9939.

[68] N. Frosst, N. Papernot, and G. Hinton, "Analyzing and improving representations with the soft nearest neighbor loss," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2012–2020.

[69] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up opensky: A large-scale ADS-B sensor network for research," in *Proc. 13th IEEE/ACM Int. Symp. Inf. Process. Sensor Netw.*, Apr. 2014, pp. 83–94.

[70] Ministry of Land, Infrastructure and Transport, South Korea, "Airportal, Electronic resource," 2024. Accessed: Apr. 7, 2023. [Online]. Available: https://www.airportal.go.kr/

[71] J. Nilsson and J. Unger, "Swedish civil air traffic control dataset," *Data Brief*, vol. 48, Jun. 2023, Art. no. 109240.

[72] X. Olive and L. Basora, "Detection and identification of significant events in historical aircraft trajectory data," *Transp. Res. Part-C, Emer. Technol.*, vol. 119, Oct. 2020, Art. no. 102737.

[73] E. Axell, P. Johansson, M. Alexandersson, and J. Rantakokko, "Estimation of the position error in GPS receivers," Swedish Defence Research Agency (FOI), Kista, Stockholm, document FOI-R–3840–SE E36044, Feb. 2014.

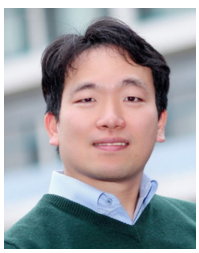
[74] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Anal. Chem.*, vol. 36, no. 8, pp. 1627–1639, Jul. 1964.
[75] D. H. Anh et al. "The UCR time series classification archive." Oct. 2018. [Online]. Available: https://www.cs.ucr.edu/eamonn/time_series_data_2018/
[76] A. Bagnall et al., "The UEA multivariate time series classification archive, 2018," 2018, *arXiv:1811.00075*.
[77] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
[78] T. Lin et al., "Microsoft COCO: Common objects in context," 2014, *arXiv:1405.0312*.
[79] M. Cuturi, "Fast global alignment kernels," in *Proc. 28th Int. Conf. Mach. Learn. (ICML)*, 2011, pp. 929–936.
[80] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 1–18.
[81] J. R. Regatti, A. A. Deshmukh, E. Manavoglu, and U. Dogan, "Consensus clustering with unsupervised representation learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–9.
[82] G. Vardakas, I. Papakostas, and A. Likas, "Deep clustering using the soft silhouette score: Towards compact and well-separated clusters," 2024, *arXiv:2402.00608*.
[83] H. Yin, A. Aryani, S. Petrie, A. Nambissan, A. Astudillo, and S. Cao, "A rapid review of clustering algorithms," 2024, *arXiv:2401.07389*.
[84] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery & Data Min.*, 2021, pp. 2114–2124.



THAWEERATH PHISANNUPAWONG received the B.Eng. degree in aeronautical engineering and commercial pilot license from the King Mongkut's Institute of Technology Ladkrabang, Bangkok, Thailand, in 2021, and the M.S. degree in aerospace engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, where he is currently pursuing the Ph.D. degree in aerospace engineering. His research focuses on aerospace data applications and representation learning.



JOSHUA J. DAMANIK (Graduate Student Member, IEEE) received the B.S. degree in engineering physics from Institut Teknologi Bandung, Indonesia, in 2018, and the M.S. degree in aerospace engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, in 2021, where he is currently pursuing the Ph.D. degree in aerospace engineering. His current research interests include robotics estimation and control, and data mining.



HAN-LIM CHOI (Senior Member, IEEE) received the B.S. and M.S. degrees in aerospace engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2000 and 2002, respectively, and the Ph.D. degree in aeronautics and astronautics from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, in 2009. Then, he studied with MIT as a Postdoctoral Associate until he joined KAIST in 2010, where he is currently a Professor of Aerospace Engineering. His research interests include estimation and control for sensor networks and decision making for multiagent systems. In 2011, he was a recipient of the Automatic Applications Prize (together with Dr. J. P. How).